



IdentityIQ® Microsoft Teams Notifications

Version: 8.5

Revised: July 2025

Copyright and Trademark Notices

Copyright © 2025 SailPoint Technologies, Inc. All Rights Reserved.

All logos, text, content, including underlying HTML code, designs, and graphics used and/or depicted on these written materials or in this Internet website are protected under United States and international copyright and trademark laws and treaties, and may not be used or reproduced without the prior express written permission of SailPoint Technologies, Inc.

"SailPoint Technologies" (design and word mark), "SailPoint" (design and word mark), "Identity IQ," "IdentityNow," "Identity AI," "Identity Cube," and "SailPoint Predictive Identity" are registered trademarks of SailPoint Technologies, Inc. "Identity is Everything," "The Power of Identity," "Navigate," and "Identity University" are trademarks of SailPoint Technologies, Inc. None of the foregoing marks may be used without the prior express written permission of SailPoint Technologies, Inc.

All other trademarks shown herein are owned by the respective companies or persons indicated.

SailPoint Technologies, Inc. makes no warranty of any kind regarding these materials or the information included therein, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. SailPoint Technologies shall not be liable for errors contained herein or direct, indirect, special, incidental or consequential damages in connection with the furnishing, performance, or use of this material.

Patents Notice: <https://www.sailpoint.com/legal/patents>

Restricted Rights Legend. All rights are reserved. No part of this document may be published, distributed, reproduced, publicly displayed, used to create derivative works, or translated to another language, without the prior written consent of SailPoint Technologies. The information contained in this document is subject to change without notice.

Use, duplication or disclosure by the U.S. Government is subject to restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 for DOD agencies, and subparagraphs (c)(1) and (c)(2) of the Commercial Computer Software Restricted Rights clause at FAR 52.227-19 for other agencies.

Regulatory/Export Compliance. The export and re-export of this software is controlled for export purposes by the U.S. Government. By accepting this software and/or documentation, licensee agrees to comply with all U.S. and foreign export laws and regulations as they relate to software and related documentation. Licensee will not export or re-export outside the United States software or documentation, whether directly or indirectly, to any Prohibited Party and will not cause, approve or otherwise intentionally facilitate others in so doing. A Prohibited Party includes: a party in a U.S. embargoed country or country the United States has named as a supporter of international terrorism; a party involved in proliferation; a party identified by the U.S. Government as a Denied Party; a party named on the U.S. Department of Commerce's Entity List in Supplement No. 4 to 15 C.F.R. § 744; a party prohibited from participation in export or re-export transactions by a U.S. Government General Order; a party listed by the U.S. Government's Office of Foreign Assets Control as ineligible to participate in transactions subject to U.S. jurisdiction; or any party that licensee knows or has reason to know has violated or plans to violate U.S. or foreign export laws or regulations. Licensee shall ensure that each of its software users complies with U.S. and foreign export laws and regulations as they relate to software and related documentation.

Contents

Introduction to IdentityIQ Microsoft Teams	1
Access Request Approval Work Item Notifications	1
Other Notifications	1
Configuring IdentityIQ Microsoft Teams	2
Prerequisites for Integrating Microsoft Teams with IdentityIQ	2
Components of IdentityIQ's Microsoft Teams	4
Best Practices for Configuring IdentityIQ Microsoft Teams	5
Creating a Connector Application in Azure	6
Creating a New Connector Group in Azure	10
Creating an SSO Entra Application Proxy in Azure	12
Creating an API Access Application in Azure	21
Creating a Microsoft Teams Application for IdentityIQ in Azure	25
Creating an Azure Active Directory Application in IdentityIQ	36
Configuring Single Sign-On to IdentityIQ from Microsoft Teams	39
Creating a Chat Application Proxy for IdentityIQ in Azure	46
Creating an Azure Bot for IdentityIQ's Microsoft Teams	49
Installing and Configuring the IdentityIQ Service Code	53
Creating a Microsoft Teams Manifest	58
Configuring API Authentication for Microsoft Teams in IdentityIQ	59
Enabling Microsoft Teams Notifications in IdentityIQ	60
Installing the IdentityIQ Application in Microsoft Teams	62
Upgrading IdentityIQ Microsoft Teams	64
Using IdentityIQ Microsoft Teams	66
Chat Tab	66
My Approvals Tab	67
Approval Tasks on Microsoft Teams	68
Auditing Microsoft Teams Notifications	69

Introduction to IdentityIQ Microsoft Teams

IdentityIQ application in Microsoft Teams allows notifications from IdentityIQ to be delivered directly to their Microsoft Teams environment, and allows end users to approve access requests on the go.

Access Request Approval Work Item Notifications

IdentityIQ now supports adaptive card notifications for Access Request Approval work items directly in Microsoft Teams. These notifications include a **My Approvals** button, which opens the *My Approvals* tab within Microsoft Teams. From there, users can review and take action on pending access requests – without needing to switch to IdentityIQ.

Activity

Chat

Teams

Calendar

Calls

OneDrive

Copilot

Home

Viva Engage

Planner

Shifts

IdentityIQ

...

IdentityIQ

Chat

My Approvals

SailPoint

3:18 pm

spadmin is requesting the following changes for **Ann Alexander**

Application	IdentityIQ
Operation	Add
Attribute	Role
Values(s)	Linebacker
Priority	Normal
Requester Comments	Test

My Approvals

spadmin is requesting the following changes for **Alice Bradley**

Application	Procurement_System
Operation	Add
Attribute	groups
Values(s)	Linebacker
Priority	Normal

My Approvals

Other Notifications

Any IdentityIQ notification can be sent as a Microsoft Teams notification and can include a link to the relevant area or action within IdentityIQ. With SAML-based Single sign-on (SSO) enabled, authenticated Microsoft Teams users can directly access IdentityIQ from links contained within the notifications, to take actions in IdentityIQ, without a separate login to IdentityIQ. For more information on IdentityIQ notifications, see the *SailPoint IdentityIQ System Configuration Guide*.

Configuring IdentityIQ Microsoft Teams

Note: Only follow these configuration instructions if you are setting up IdentityIQ application in Microsoft Teams for the first time. If you are upgrading an existing installation, refer to the [Upgrading IdentityIQ Microsoft Teams](#) guide for the appropriate steps.

Integrating Microsoft Teams with IdentityIQ requires setting up and configuring components in Azure, as well as configuring IdentityIQ to support Teams-based notifications and Access Request Approvals work items.

Note: Some configuration steps rely on information provided by earlier steps. Therefore, we strongly recommend following the integration process in the order outlined in this guide to ensure a smooth and successful setup.

Configuring SAML-based single Sign-on (SSO) between Azure and IdentityIQ is a mandatory step in this process. Enabling SSO between IdentityIQ and Azure gives users a seamless experience using notifications in Microsoft Teams. It removes the need to log in separately to IdentityIQ or to browse to the correct IdentityIQ location to take necessary action on the notifications that includes a link.

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

Prerequisites for Integrating Microsoft Teams with IdentityIQ

Prerequisites for integrating the Microsoft Teams integration for IdentityIQ include the following:

Supported Operating Systems

IdentityIQ application in Microsoft Teams is supported on any currently available and supported versions of Windows and Linux.

Azure Prerequisites

- Users should have a Microsoft Entra Subscription (for Entra Application Proxy) and Azure Subscription (for Azure Bot Service) along with a basic understanding of the Azure portal to configure the proxy and related services within their tenant.

- A configured and active Azure tenant for your organization with users assigned the following roles:
 - **Application Administrator** – enables the users to create and manage all aspects of application registrations and enterprise applications.
 - **Cloud Application Administrator** – similar to Application Administrator but cannot manage applications.
 - **Teams Administrator** – enables the users to manage IdentityIQ's Microsoft Teams services.
 - **Teams Device Administrator** – enables the users to perform management related tasks on Microsoft Teams certified devices.
 - **User Administrator** – enables the users to manage all aspects of users and user group, including resetting passwords for limited administrators.
- A Microsoft Teams account with permissions to install applications.
- Users should be familiar with the fundamentals of SAML Single sign-on (SSO) setting.
- *Optional:* An Azure resource group dedicated to your IdentityIQ integration, which can help organize and manage related resources more effectively.

IdentityIQ Prerequisites

- *Recommended:* Use a separate private server to run the IdentityIQ service code. The IdentityIQ service code should **not** be hosted on the same server as the main IdentityIQ application. The service code needs to be accessible from the internet whereas IdentityIQ server itself should remain isolated and not exposed to the internet.

Connectivity and Security Prerequisites

- A network connection that enables Microsoft Teams to communicate with the IdentityIQ service code. This requires a public message endpoint that routes traffic to the private IP of the server hosting the service code.
- To ensure Microsoft Teams can trust calls to the IdentityIQ service code, it must support HTTPS and use a valid certificate issued from a Microsoft supported Certificate Authority for the specified domain name. This certificate and its corresponding key are utilized by the IdentityIQ service code running on the private server.

Note: Microsoft Teams will send messages to a server with valid SSL certificate. However, if your SSL certificate is invalid, configure the Azure Application to accept the invalid certificate and send messages. See [Creating a Chat Application Proxy for IdentityIQ in Azure](#) for details.

Components of IdentityIQ's Microsoft Teams

Here is an overview of the components that are part of the IdentityIQ Microsoft Teams Notifications and Access Request Approvals feature. Some of these components are created as **App registrations** in Azure. Detailed instructions for creating and configuring these components are provided later in this document.

SSO Entra Application Proxy – An Azure application that acts as a proxy for accessing the IdentityIQ without configuring firewall for public internet access. The application enables the IdentityIQ approvals page to load with Single sign-on (SSO) within Microsoft Teams. It should be created under the Enterprise Applications section in Azure. This setup facilitates proxy configuration and SSO integration for accessing *My Approvals* page of IdentityIQ/UI in Microsoft Teams. For detailed steps, see [Creating an SSO Entra Application Proxy in Azure](#).

Connector Application – Register a Connector application in Azure and then create a corresponding Azure Active Directory (AD) application in IdentityIQ using the credentials from the Azure app registration. For detailed steps, see [Creating a Connector Application in Azure](#).

API Access Application – An Azure app registration that facilitates token-based authentication between Microsoft Teams and IdentityIQ by defining a GetToken scope and the appropriate audience value. When an API token request is made, IdentityIQ will check for specific variables to verify the token and, upon successful validation, issues a user-specific Access Token for the bot to use. For detailed steps, see [Creating an API Access Application in Azure](#).

Teams Application – An Azure app registration that defines the scope required for IdentityIQ's Microsoft Teams integration. Values from this application will be needed when creating Azure bot. The values will also be needed for the .env file used to configure the bot service code. For detailed steps, see [Creating a Microsoft Teams Application for IdentityIQ in Azure](#).

Chat Application Proxy – An Azure application for the bot API / message endpoint. For detailed steps, see [Creating a Chat Application Proxy for IdentityIQ in Azure](#).

Azure Bot for Microsoft Teams – An Azure bot that utilizes Microsoft's artificial intelligence (AI)-powered chatbot to facilitate communication between Microsoft Teams and IdentityIQ. Additionally, this bot also stores the necessary configuration to access the IdentityIQ service code. For detailed steps, see [Creating an Azure Bot for IdentityIQ's Microsoft Teams](#).

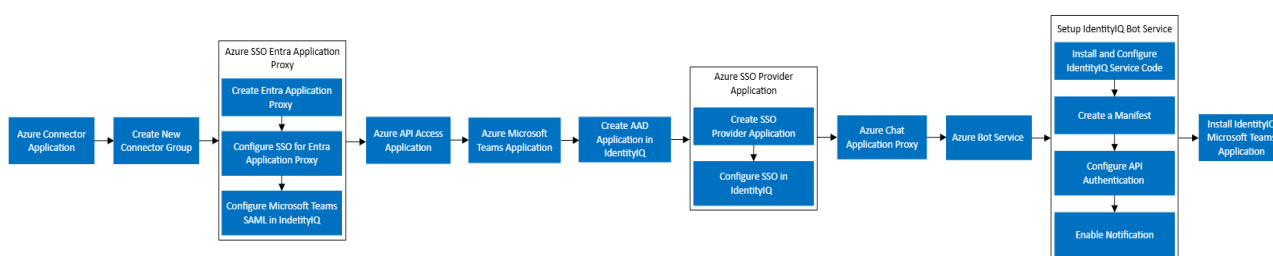
IdentityIQ service code – This code is provided by SailPoint and is installed in your environment. The service code package includes an environment (.env) file that must be configured with essential resource information such as the Azure Teams app ID, Azure Teams app secret, Azure tenant ID, and other relevant details. After configuring the .env file, a manifest can be generated, which is used to deploy the IdentityIQ application within the user's Microsoft Teams environment. For detailed steps, see [Installing and Configuring the IdentityIQ Service Code](#).

SSO Provider – An Azure app registration that facilitates SAML-based Single sign-on (SSO) between Microsoft Teams and IdentityIQ. Enabling this option simplifies the login process for Microsoft Teams users by allowing them to bypass the IdentityIQ login when accessing the IdentityIQ links in Microsoft Teams notification. For more details, see [Configuring Single Sign-On to IdentityIQ from Microsoft Teams](#).

Best Practices for Configuring IdentityIQ Microsoft Teams

SailPoint recommends following these best practices when implementing the IdentityIQ Microsoft Teams Notifications and Access Request Approvals feature:

- Follow the flowchart below carefully and create the Azure applications in the specified sequence to ensure a smooth configuration of the IdentityIQ Microsoft Teams with ease.



- Refer to the table below for the configuration sequence, suggested Azure application name, and reference guides needed to set up the IdentityIQ application within the Microsoft Teams environment.

Sequence Number	Suggested Azure Name	Reference Guide
1	IdentityIQ Connector	Creating a Connector Application in Azure
2	Not Applicable	Creating a New Connector Group in Azure
3	IdentityIQ SSO Entra Proxy	Creating an SSO Entra Application Proxy in Azure
4	IdentityIQ API Access	Creating an API Access Application in Azure
5	IdentityIQ Teams	Creating a Microsoft Teams Application for IdentityIQ in Azure
6	Not Applicable	Creating an Azure Active Directory Application in IdentityIQ
7	IdentityIQ SSO Provider	Configuring Single Sign-On to IdentityIQ from Microsoft Teams
8	IdentityIQ Chat Proxy	Creating a Chat Application Proxy for IdentityIQ in Azure

Sequence Number	Suggested Azure Name	Reference Guide
9	IdentityIQ <MyCompany> Bot	Creating an Azure Bot for IdentityIQ's Microsoft Teams
10	Not Applicable	Installing and Configuring the IdentityIQ Service Code Creating a Microsoft Teams Manifest Configuring API Authentication for Microsoft Teams in IdentityIQ Enabling Microsoft Teams Notifications in IdentityIQ
11	Not Applicable	Installing the IdentityIQ Application in Microsoft Teams

Creating a Connector Application in Azure

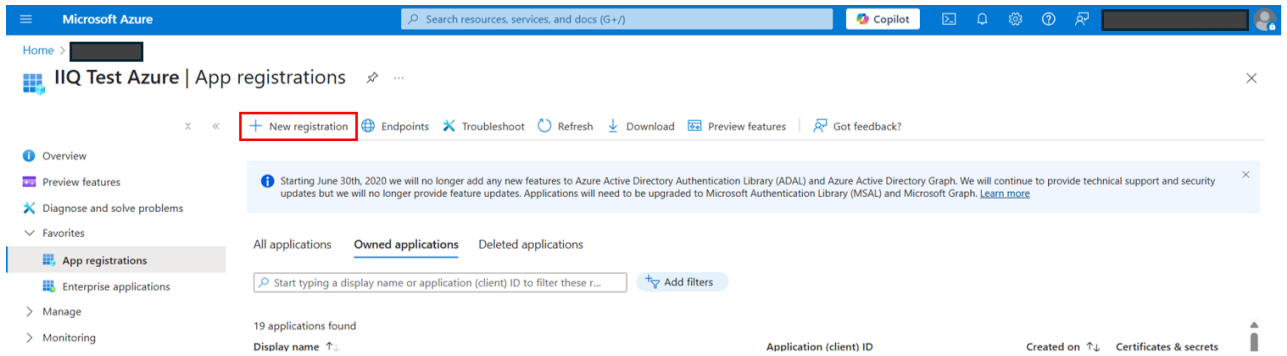
The Connector application facilitates the integration of Azure Active Directory user and group data into IdentityIQ by providing the necessary permissions. It establishes communication with the corresponding Azure Active Directory application within IdentityIQ to initiate workflows for Access Request Approvals. These workflows are triggered using data, events, and resources within IdentityIQ.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

To register a connector application, perform the following steps:

1. Sign in to Azure portal and navigate to your **Microsoft EntraID**.
2. Under **Manage > App registrations**, select **+ New registration**.

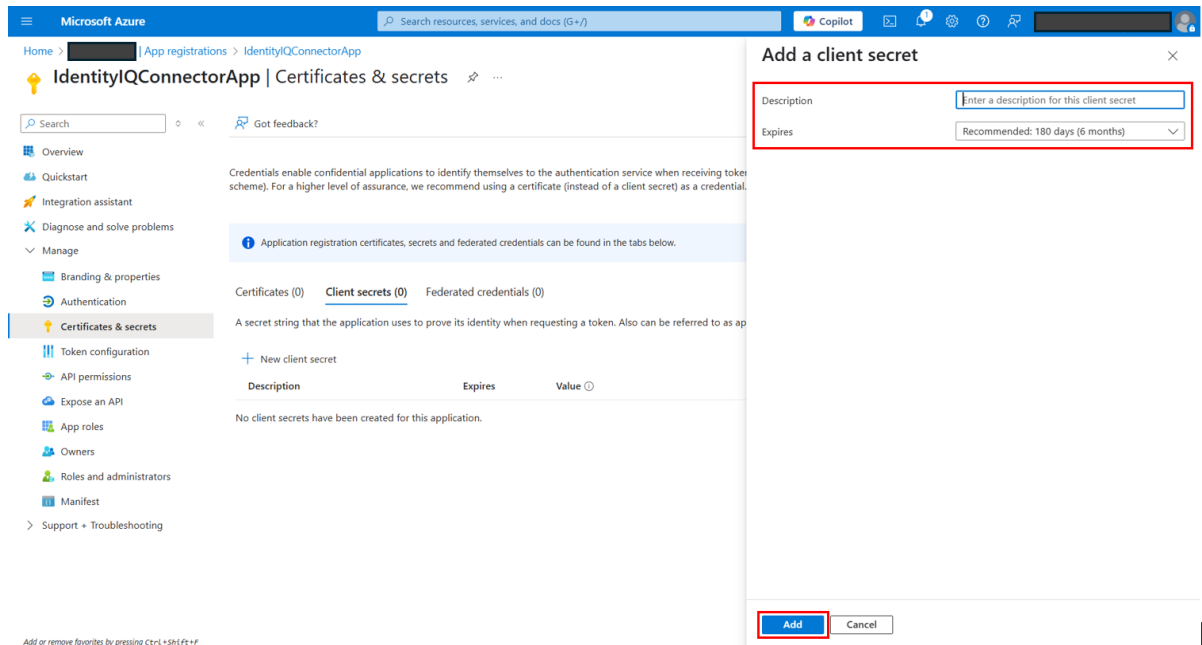


3. On the Registration page, provide the following details:

- Name** – Provide a meaningful name for the application. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.
- Supported account types** – Select **Single tenant only**.
- Redirect URL (Optional)** – Leave this field blank.
- Select **Register**.

4. Create a client secret for the application:

- a. In the left navigation, select **Manage > Certificates and secrets**.
- b. Under the **Client secrets** tab, select **+ New client secret**.
- c. In the dialog that displays:

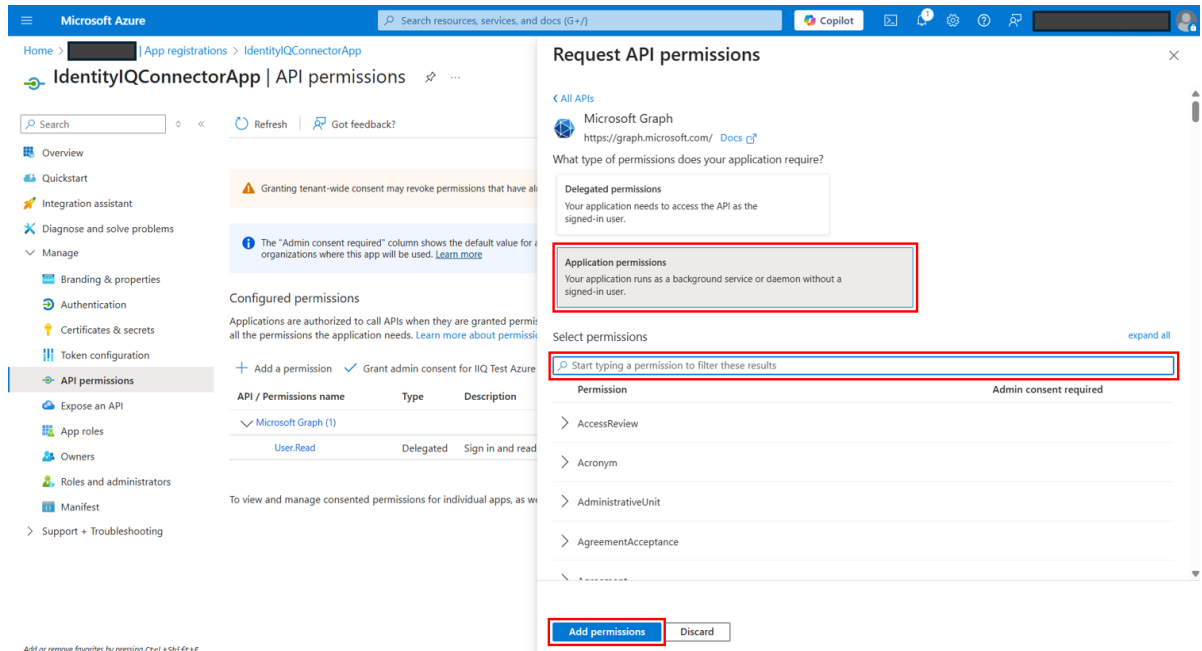


- i. Provide a **Description** for the secret.
- ii. Select a time of expiration in **Expires** field.
- iii. Select **Add**.

Note: Make a note of the Client Secret Value and Client Secret ID. These will be required when creating the Azure Active Directory application in IdentityIQ. See [Creating an Azure Active Directory Application in IdentityIQ](#) for details.

5. Configure API permissions:

- a. In the left navigation, select **API permissions**.
- b. Select **+ Add a permission**.
- c. In the dialog that displays:



- i. Under the **Microsoft APIs** tab, select **Microsoft Graph**.
- ii. Select **Application permissions**.
- iii. Under **Select permissions**, search and add the following:
 - Application.Read.All
 - AppRoleAssignment.ReadWrite.All
 - Directory.ReadWrite.All
 - Group.Read.All
 - Group.ReadWrite.All
 - IdentityRiskEvent.Read.All
 - IdentityRiskyUser.Read.All
 - Organization.Read.All

- RoleManagement.ReadWrite.Directory
- User.Invite.All
- User.Read.All
- User.ReadWrite.All

iv. After selecting all permissions, select **Add permissions**.

Note: Ensure that all the above permissions are successfully added. These permissions are essential for the application to interact properly with IdentityIQ.

Once the permissions are granted, you will see the below screen.

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission ✓ Grant admin consent for IIQ Test Azure

API / Permissions name	Type	Description	Admin consent requ...	Status
▼ Microsoft Graph (12) ...				
Application.Read.All	Application	Read all applications	Yes	✓ Granted for IIQ Test Azu ...
AppRoleAssignment.ReadWri	Application	Manage app permission grants and app role assignments	Yes	✓ Granted for IIQ Test Azu ...
Directory.ReadWrite.All	Application	Read and write directory data	Yes	✓ Granted for IIQ Test Azu ...
Group.Read.All	Application	Read all groups	Yes	✓ Granted for IIQ Test Azu ...
Group.ReadWrite.All	Application	Read and write all groups	Yes	✓ Granted for IIQ Test Azu ...
IdentityRiskEvent.Read.All	Application	Read all identity risk event information	Yes	✓ Granted for IIQ Test Azu ...
IdentityRiskyUser.Read.All	Application	Read all identity risky user information	Yes	✓ Granted for IIQ Test Azu ...
Organization.Read.All	Application	Read organization information	Yes	✓ Granted for IIQ Test Azu ...
RoleManagement.ReadWrite.	Application	Read and write all directory RBAC settings	Yes	✓ Granted for IIQ Test Azu ...
User.Invite.All	Application	Invite guest users to the organization	Yes	✓ Granted for IIQ Test Azu ...
User.Read.All	Application	Read all users' full profiles	Yes	✓ Granted for IIQ Test Azu ...
User.ReadWrite.All	Application	Read and write all users' full profiles	Yes	✓ Granted for IIQ Test Azu ...

You now have successfully created a Connector application in Azure. For next step, refer [Creating a New Connector Group in Azure](#).

Creating a New Connector Group in Azure

Note: Create a new connector group in Azure only if there is no existing connector group in your virtual machine (VM) that can be assigned to Azure Entra Application Proxy. See [Creating an SSO Entra Application Proxy in Azure](#) for more details.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

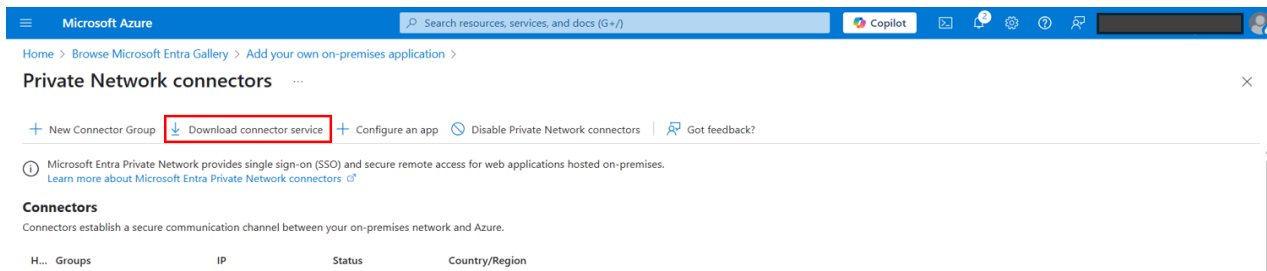
Perform the following steps to install a new connector group:

1. Sign in to Azure portal and navigate to your **Microsoft Entra ID**.
2. On the Microsoft Entra Gallery page, select **Add on-premises application**.
3. Select the link on the page to download a connector or manage connector group.

The screenshot shows the 'Add your own on-premises application' page in the Microsoft Azure portal. The page has a blue header with the Microsoft Azure logo and a search bar. Below the header, there's a breadcrumb trail: 'Home > Browse Microsoft Entra Gallery >'. The main heading is 'Add your own on-premises application'. There are several tabs: 'Basic' (selected) and 'Advanced'. Under the 'Basic' tab, there are several configuration options: 'Maintenance mode' (checkbox), 'Name' (text input), 'Internal Url' (text input with 'https://' pre-filled), 'External Url' (text input with 'https://' pre-filled and a dropdown menu showing '-iiqtest.msappproxy.net/'), 'Application segments' (text input with 'Add application segments'), 'Pre Authentication' (dropdown menu showing 'Microsoft Entra ID'), 'Connector Group' (dropdown menu showing 'Default - North America'), and 'SSL Certificate' (text input with 'No SSL certificate required.'). At the bottom of the page, there's a blue box with a message: 'We recommend at least two active connectors in selected group 'Default''. Below this message is a link: 'Click here to download a connector or manage your connector groups.' This link is highlighted with a red box.

Once the link is selected, Private Network connectors page will be displayed.

4. Select **Download connector service**.



5. Install the downloaded .exe file in VM.

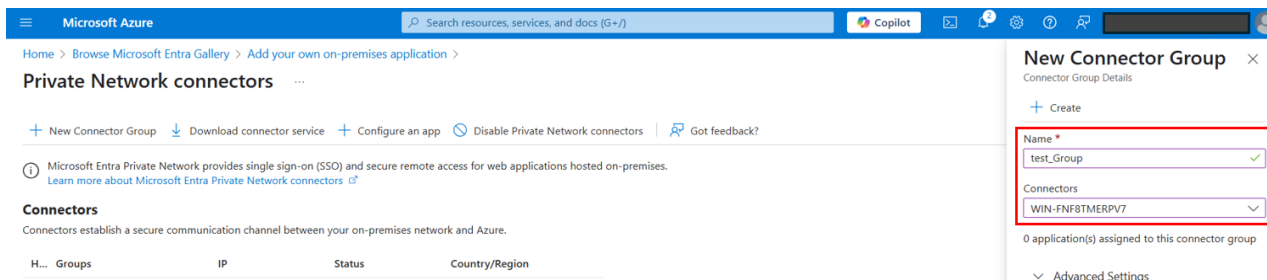
Note: This .exe file should be deployed on a VM where IdentityIQ has not been deployed. However, both the VMs should be within the same network.

6. During installation, you will be prompted for authentication. Complete the authentication process and select **OK**.

Once the installation is successful, the connector will appear under the Default Connector Group.

7. Select **+ New Connector Group** and then select your VM.

8. Add **Name** and **Connectors**.



9. Select **+ Create**.

You now have successfully created a new connector group in Azure. For next step, refer [Creating an SSO Entra Application Proxy in Azure](#).

Creating an SSO Entra Application Proxy in Azure

Azure Entra Application Proxy is a cloud-based service that enables secure remote access to IdentityIQ within the Microsoft Teams application. It allows assigned users and groups to access IdentityIQ from anywhere – without the need of virtual private network (VPN), ensuring a seamless and secure experience.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a

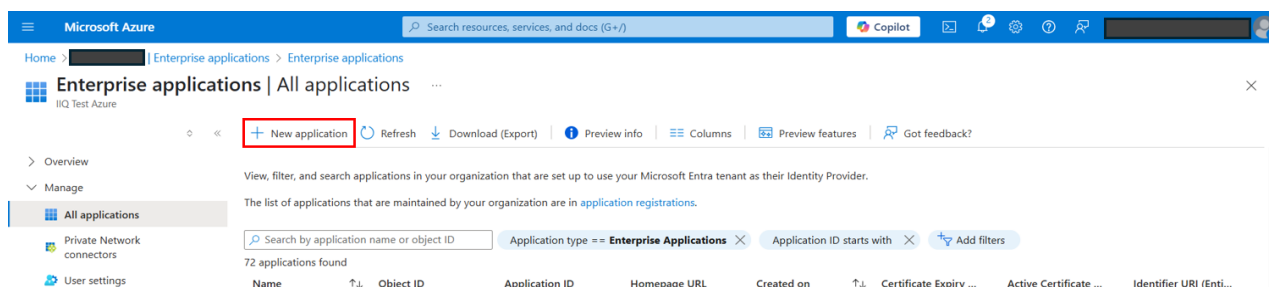
Visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

Creating Entra Application Proxy in Azure

To register an Entra Application Proxy, perform the following steps:

1. Under **Manage > Enterprise applications**, select **+ New application**.



2. On the Microsoft Entra Gallery page, select **Add on-premises application**.

3. On the Add application page, provide the following details:

a. Under **Basic** tab:

- i. **Maintenance mode** – Keep it unselected. This is the maintenance page that users see during maintenance and downtime events.
 - ii. **Name** – Provide a meaningful name for the application. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.
 - iii. **Internal Uri** – This is the IdentityIQ URL the proxy will forward traffic to. The URL should include the VM name, port, context, and *<My Approvals>* tab of your IdentityIQ application. For example: `https://<VM NAME>:<port>/identityiq/ui/`.
 - iv. **External Uri** – An auto-generated external URL with application name options as input. The URL can be customized. For example: `https://<APP_NAME>.msproxy.net/identityiq/ui/`.
- Note: External URL only exposes IdentityIQ Approvals page to users accessing from the Microsoft Teams application (native app or web). Users cannot access any other pages using this URL, ensuring secure access.
- v. **Pre Authentication** – Attribute for pre authentication. By default, Microsoft Entra ID is selected. Keep the default selection.
 - vi. **Connector Group** — Select your connector group appropriate for your IdentityIQ VM from the dropdown.

- b. Under **Advance** tab on the Application Proxy page, ensure that **Validate Backend SSL certificate** checkbox is selected by default. But unselect the checkbox if the certificate for IdentityIQ application server is expired, self-generated, or otherwise invalid.

The screenshot shows the 'Add your own on-premises application' page in the Microsoft Azure portal. The 'Advanced' tab is selected. The 'Validate Backend SSL certificate' checkbox is checked and highlighted with a red box. The 'Create' button at the bottom is also highlighted with a red box.

Microsoft Azure

Search resources, services, and docs (G+/I)

Copilot

Home > Enterprise applications > Enterprise applications | All applications > Browse Microsoft Entra Gallery >

Add your own on-premises application

Got feedback?

Introducing Application segments for wild card applications - a simpler way to publish complex distributed applications made up of multiple regular apps and manage their CORS settings. [Learn more](#)

Private Network provides single sign-on (SSO) and secure remote access for web applications hosted on-premises. [Learn more](#)

Basic **Advanced**

Backend Application Timeout

Use Http-Only Cookie ☐

Use Persistent Cookie ☐

Translate Urls in headers ☒

Translate Urls in application body ☐

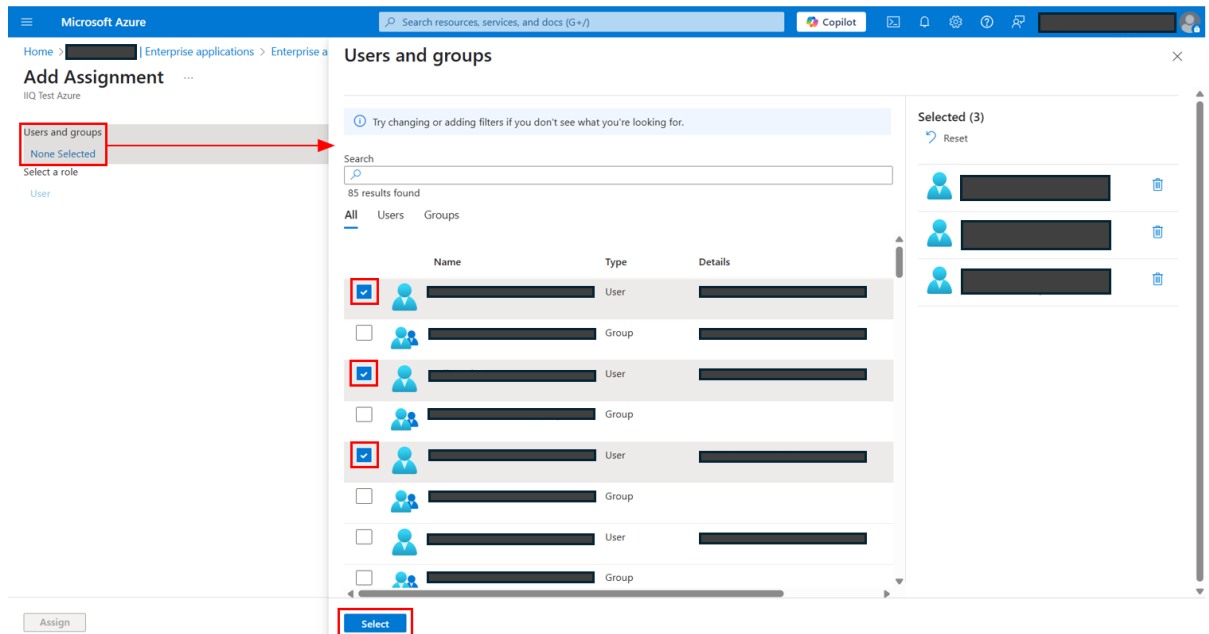
Validate Backend SSL certificate ☒

Create Discard

- c. Select **Create**.

4. Select **Create**.
5. Under **Manage > Enterprise applications**, search and open the newly created application proxy.
6. On Overview page, select **Assign users and groups** tile.
- Alternatively, navigate to **Manage > Users and groups** and then select **+ New user/group**.

- a. On Add Assignment page, under Users and groups, select **None Selected**.



- b. On the Users and groups dialog, provide the following details:
- Select the users and groups that you want to be able to use SAML SSO.
 - Select the **Select** button.
- c. Select **Assign**.

Configuring Single Sign-On for Entra Application Proxy

After creating an Entra Application Proxy and adding users and groups to it, you need to configure the Single sign-on (SSO) for the application.

- In the left navigation panel, select **Single sign-on**.
- Select the **SAML** tile.
- On the Set up page, select **Edit** for Basic SAML Configuration.
- In the dialog, provide the following details:

Basic SAML Configuration

Identifier (Entity ID) *

Reply URL (Assertion Consumer Service URL) *

Sign on URL (Optional)

Relay State (Optional)

Logout URL (Optional)

Attributes & Claims

Unique User Identifier

SAML Certificates

- Add Identifier** – Add the auto generated external URL during the application proxy creation. Ignore this step, if the URL is already added by default.
 - Add reply URL** – Add the auto generated external URL during application proxy creation with suffix; **/index.jsf?msTeamsAuthComplete=true**. For example, **https://SSOEntraProxy-test.m-sappproxynet/identityiq/ui/index.jsf?msTeamsAuthComplete=true**.
 - Under **Sign on URL (Optional)**, add the auto generated external URL.
 - Do not add a URL for **Relay State (Optional)** and **Logout URL (Optional)**.
 - Select **Save**.
- On the Set up page select **Edit** for Attributes and Claims.
 - On the Attributes & Claims page, select **Unique User Identifier (Name ID)**.
 - On the Manage claim page, provide the following details:

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > Enterprise applications > Enterprise applications | All applications > IdentityIQSSOEntraAppProxy | SAML-based Sign-on > SAML-based Sign-on > Attributes & Claims >

Manage claim

Save Discard changes Got feedback?

Name: nameidentifier

Namespace: http://schemas.xmlsoap.org/ws/2005/05/identity/claims

Choose name identifier format

Name identifier format * Unspecified

Source * ☒ Attribute ☐ Transformation ☐ Directory schema extension

Source attribute * user.objectid

Claim conditions

Advanced SAML claims options

- Select **Unspecified** for Name identifier format.
- Select **user.objectid** for Source attribute.
- Select **Save**.

8. On the Single sign on page, perform the following steps:

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > Enterprise applications > Enterprise applications | All applications > IdentityIQSSOEntraAppProxy >

IdentityIQSSOEntraAppProxy | SAML-based Sign-on

Enterprise Application

Overview Deployment Plan Diagnose and solve problems Manage Properties Owners Roles and administrators Users and groups Single sign-on Provisioning Application proxy Self-service Custom security attributes Security Activity Troubleshooting + Support

Upload metadata file Change single sign-on mode Test this application Got feedback?

Status: Active

Thumbprint: [redacted]

Expiration: 5/29/2028, 1:08:48 PM

Notification Email: [redacted]

App Federation Metadata Url: [redacted]

Certificate (Base64) Download

Certificate (Raw) Download

Federation Metadata XML Download

Verification certificates (optional) Edit

Required: No

Active: 0

Expired: 0

4 Set up IdentityIQSSOEntraAppProxy

You'll need to configure the application to link with Microsoft Entra ID.

Login URL: [redacted]

Microsoft Entra Identifier: [redacted]

Logout URL: [redacted]

5 Test single sign-on with IdentityIQSSOEntraAppProxy

Test to see if single sign-on is working. Users will need to be added to Users and groups before they can sign in.

Test

- Download **Certificate (Base64)**.
- Save the certificate in your local drive and use it during IdentityIQ configuration in the next section.

- c. Copy **Login URL** and **Microsoft Entra Identifier** URL to use it during application configuration to link with Microsoft Entra ID in the next section.

Now configure the Microsoft Teams SAML in the Debug page.

Configuring Microsoft Teams SAML in Debug Page

To update the Microsoft Teams SAML configuration in the debug page, perform the following steps:

1. Log in as an IdentityIQ administrator.
2. Navigate to the Debug page through the Debug URL: `https://<hostname>/identityiq/debug`.
3. Select **Configuration** object and then select **System Configuration**. The Object Editor will open, showing the configuration settings in XML format.
4. Search for "TeamsSAMLEnabled" key in the XML and set to TRUE in the following manner; `<entry key="teamsSAMLEnabled" value="true"/>`.
5. Select **Save**.
6. In the Filter by Name or ID search field, type '**TeamsSAML**' and press ENTER. Select the TeamsSAML row to open the Object Editor.
7. Add IdentityNow attribute in TeamsSAML configuration.
8. Add below snippet into configuration tag.

```
<Attributes>
  <Map>
    <entry key="IdentityNow">
      <value>
        <SAMLConfig assertionConsumerService="ENTRA-PROXY-URLindex.jsf"
bindingMethod="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST" entityId="ENTRA-
PROXY-URL" idpServiceUrl="MICROSOFT-LOGIN-URL" issuer="MICROSOFT-ENTRA-
IDENTIFIER" nameIdFormat="urn:oasis:names:tc:SAML:1.1:nameid-form-
at:unspecified">
          <IdpPublicKey>SAML-CERTIFICATE</IdpPublicKey>
          <RuleRef>
            <Reference class="sailpoint.object.Rule" id="RULE-ID"
name="ExampleAzureActiveDirectoryUsingTenantIdSAML"/>
          </RuleRef>
        </SAMLConfig>
      </value>
    </entry>
  </Map>
</Attributes>
```

9. To add attribute IdentityNow in TeamsSAML configuration, you will need the following values:

- a. ENTRA-PROXY-URL
- b. MICROSOFT-LOGIN-URL
- c. MICROSOFT-ENTRA-IDENTIFIER
- d. SAML-CERTIFICATE Public Key (IdP - Identity Provider)
- e. RULE-ID

10. The above values can be found as mentioned below:

- a. ENTRA-PROXY-URL, MICROSOFT-LOGIN-URL, MICROSOFT-ENTRA-IDENTIFIER; navigate to **Microsoft Entra ID > Enterprise Application** in Azure, then open the Entra Application Proxy created earlier.
 - ENTRA-PROXY-URL = External URL
 - MICROSOFT-LOGIN-URL = Login URL
 - MICROSOFT-ENTRA-IDENTIFIER = Microsoft Entra Identifier
- b. SAML-CERTIFICATE Public Key; use the public key from the SAML **Certificate (Base64)** downloaded during the configuration process.
- c. RULE-ID; navigate to IdentityIQ > Debug > Rule > ExampleAzureActiveDirectoryUsingTenantIdSAML and copy the Id from that rule and use it as the RULE-ID in the configuration.

Note: You may have already copied some of these values when setting up Single sign-on (SSO) for the Entra Application Proxy.

11. If the ENTRA-PROXY-URL already contains a trailing slash, as is typically the case, refrain from adding an additional slash. For example, if the Entra-Proxy-URL is `https://abc/`, then `assertionConsumerService` should be `https://abc/index.jsf`, without an additional slash before `index.jsf`.

If it doesn't have a trailing slash, for example; `https://abc`, then add one slash before `index.jsf`, for example; `https://abc/index.jsf`.

You now have successfully created an SSO Entra Application Proxy in Azure. For next step, refer [Creating an API Access Application in Azure](#).

Creating an API Access Application in Azure

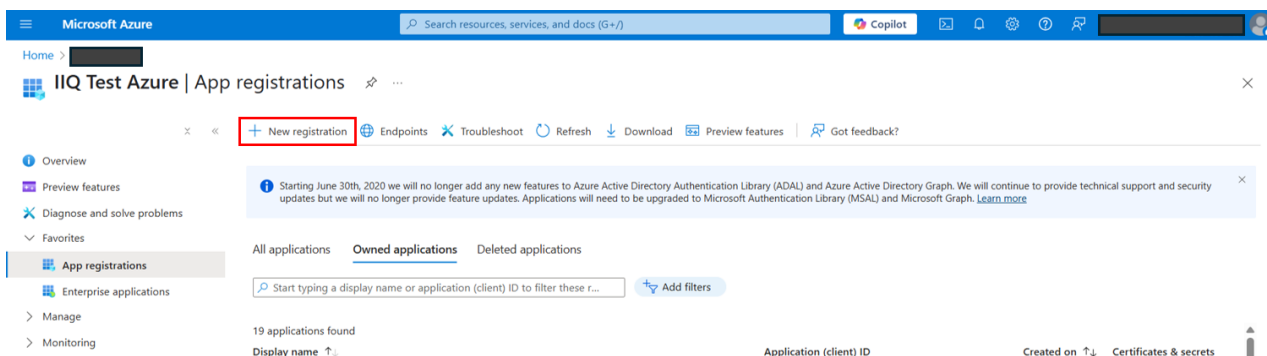
To enable token-based authentication between Microsoft Teams and IdentityIQ, you must create an API access application in Azure, then use some of the values from the API application within IdentityIQ, to enable authentication. It includes the GetToken scope and the proper audience value. When IdentityIQ received an API token request, it checks for specific values to validate the token. If the token is valid, IdentityIQ generates an Access Token for the user, which is then used by the bot to perform its task.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

To create the API access application in Azure, perform the following steps:

1. Sign into Azure portal and navigate to your **Microsoft Entra ID**.
2. Under **Manage > App registrations**, select **+ New registration**.

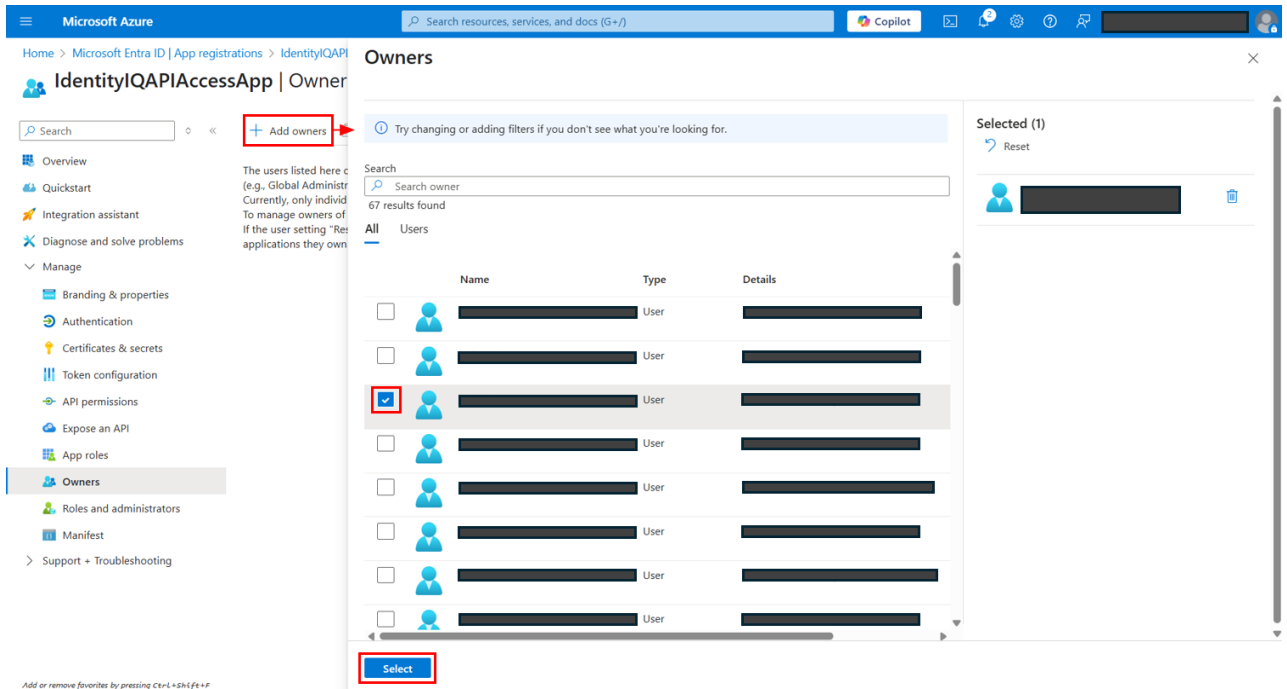


3. On the Registration page, provide the following details:

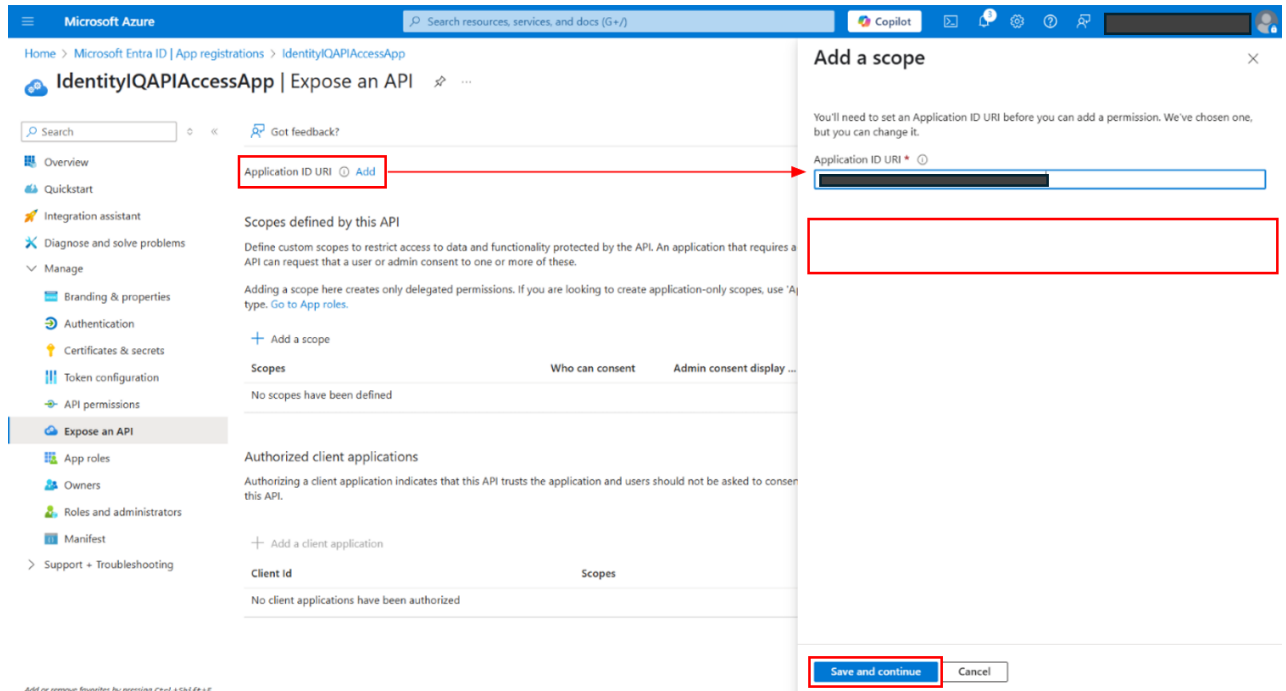
The screenshot shows the 'Register an application' page in the Microsoft Azure portal. The page has a blue header with the 'Microsoft Azure' logo and a search bar. Below the header, the breadcrumb 'Home > App registrations' is visible. The main heading is 'Register an application'. The form includes a 'Name' field with a red box around it, a 'Supported account types' dropdown menu with 'Single tenant only' selected (also highlighted with a red box), and a 'Register' button at the bottom (highlighted with a red box). The page also contains a 'Help me choose' link and a 'By proceeding, you agree to the Microsoft Platform Policies' checkbox.

- a. **Name** – Provide a meaningful name for the application. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.
- b. **Supported account types** – Select **Single tenant only**.
- c. Select **Register**.

4. Create an owner for the application:



- a. In the left navigation, select **Owners**.
 - b. Select **+ Add owners**.
 - c. Search and select your name or email.
 - d. Select the **Select** button.
5. In the left navigation, select **Expose an API**.
 6. Select **Application ID URI**. The Application ID URI field is populated in the format; `api://<IdentityIQ API Access APP ID>`.

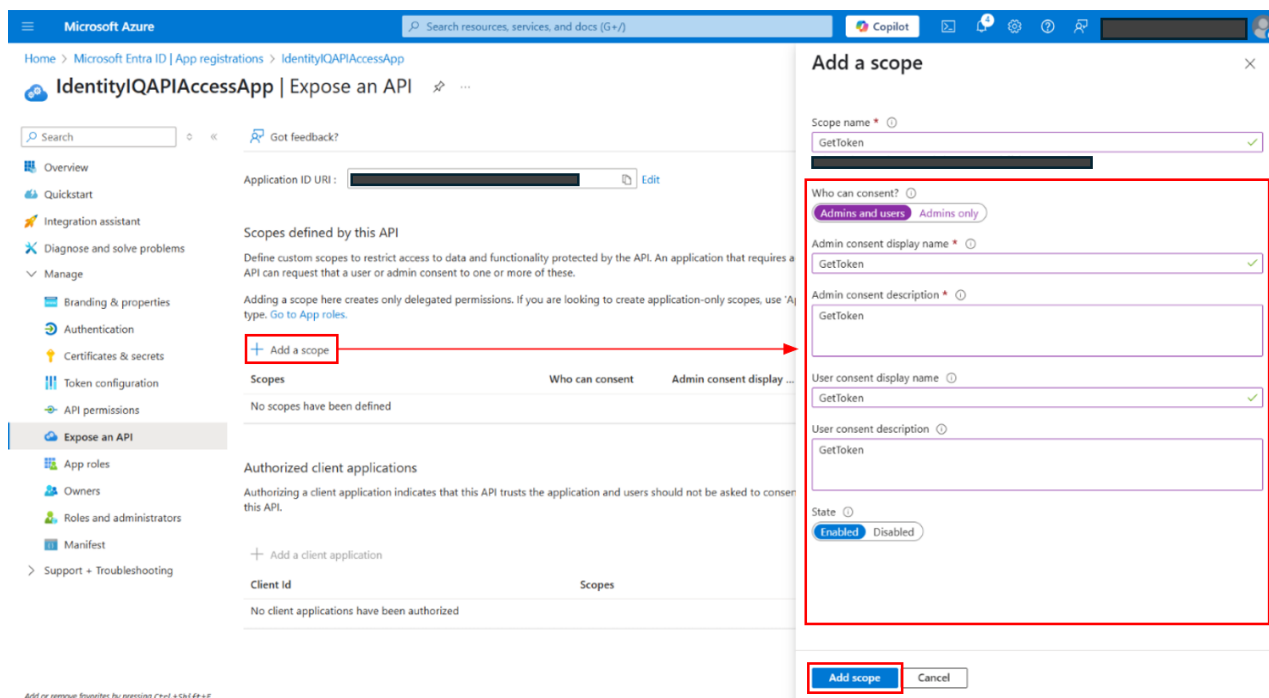


7. Add `/identityiq/api` to it so it becomes; `api://<IdentityIQ API Access APP ID>/identityiq/api`.

Note: You can add any other suffix to the Application ID URI. Make sure to use the same suffix during API Token Settings in [Configuring API Authentication for Microsoft Teams in IdentityIQ](#).

8. Select **Save and Continue**.

9. Add a GetToken scope:



- a. On the Expose an API page, select **+ Add a scope**.
- b. In the dialog that displays:
 - i. Scope name – Type **GetToken**.
 - ii. Who can consent – Select **Admins and users** option.
 - iii. Add display names and descriptions for **Admin consent** and **User consent**.
 - iv. Select **Enabled** state.
 - v. Select **Add Scope**.

You now have successfully created an API Access application in Azure. For next step, refer [Creating a Microsoft Teams Application for IdentityIQ in Azure](#).

Creating a Microsoft Teams Application for IdentityIQ in Azure

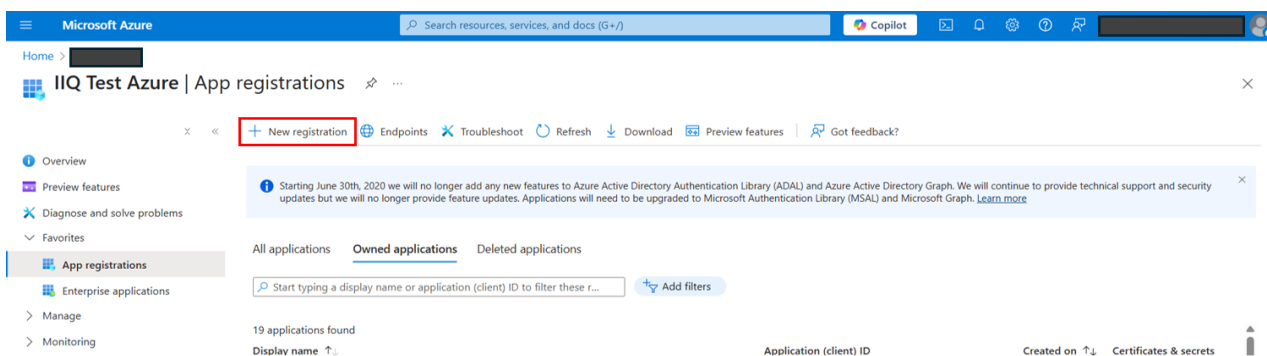
The Microsoft Teams application for IdentityIQ in Azure specifies the scope utilized by the IdentityIQ Microsoft Teams feature. Within this application, you configure authentication parameters, establish a client secret, and select the appropriate Microsoft API permissions. This application will be associated with the Azure bot resource. Values from this application will also be needed for the .env file used to configure the IdentityIQ service code.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

Perform the following steps to create a Microsoft Teams application for IdentityIQ:

1. Sign into **Azure** portal and navigate to your **Microsoft Entra ID**.
2. Under **Manage > App registrations**, select **+ New registration**.



3. On the Registration page, provide the following details:

The screenshot shows the 'Register an application' page in the Microsoft Azure portal. The page has a blue header with the 'Microsoft Azure' logo and a search bar. Below the header, there's a breadcrumb trail: 'Home > App registrations > Register an application'. The main content area contains several form fields: a 'Name' field with a red asterisk and a description 'The user-facing display name for this application (this can be changed later)'; a 'Supported account types' dropdown menu with 'Single tenant only' selected; a 'Redirect URI' field with a placeholder 'e.g. https://example.com/auth'; and a 'Register' button at the bottom. Red boxes highlight the 'Name' field, the 'Supported account types' dropdown, and the 'Register' button.

- a. **Name** - Type a name for Microsoft Teams application for IdentityIQ. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.
- b. **Supported account types** – Select **Single tenant only**.
- c. **Redirect URL (Optional)** – Leave this field blank.
- d. Select **Register**.

Note: Once created, overview details of the application including Display name, Application (client) ID, Object ID, Directory (tenant) ID etc. will be displayed. Copy and save Application (client) ID for use during [Creating an Azure Bot for IdentityIQ's Microsoft Teams](#), [Creating an Azure Active Directory Application in IdentityIQ](#), and [Installing and Configuring the IdentityIQ Service Code](#).

4. Configure an Authentication redirect URI:
 - a. In the left navigation, select **Authentication**.
 - b. Select **+ Add a platform**.
 - c. On the Configure platforms dialog, select **Web**.

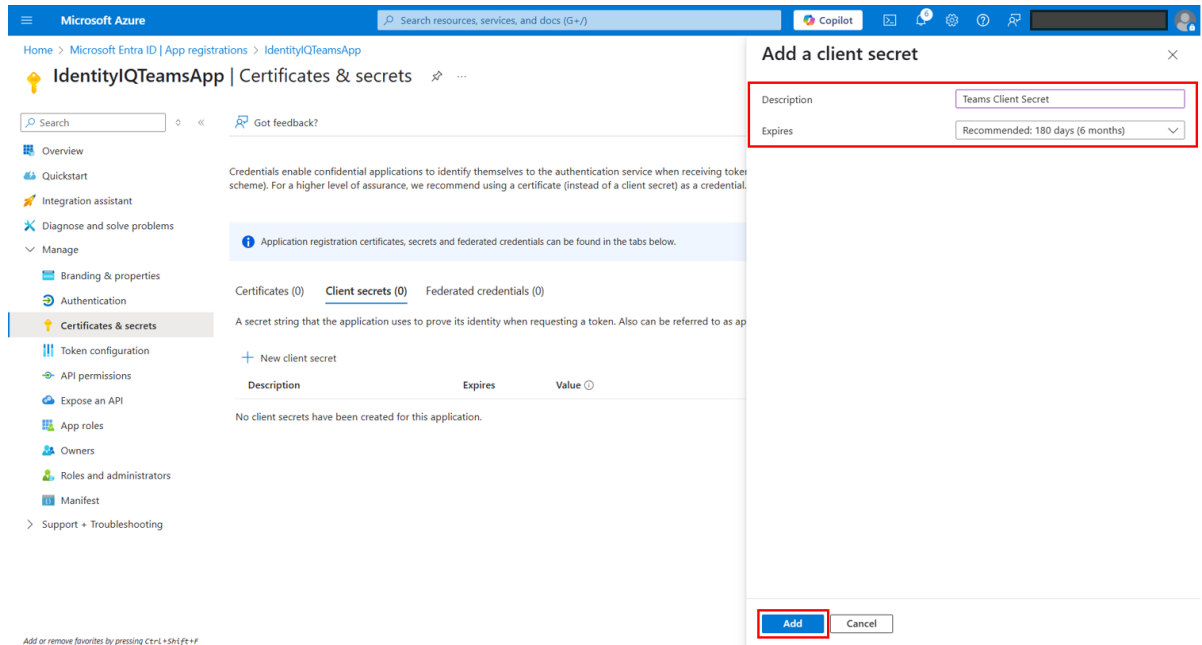
- d. On the Configure Web dialog, provide the following details:

The screenshot shows the Microsoft Azure portal interface. On the left, the navigation pane is visible with the 'Authentication' section selected. The main area displays the 'IdentityIQTeamsApp | Authentication' page. The 'Platform configurations' section is active, showing 'Supported account types' and 'Advanced settings'. A 'Configure Web' dialog is open on the right, with the 'All platforms' tab selected. The dialog contains the following sections:

- Redirect URIs:** A text input field with a green checkmark, indicating a valid URI has been entered.
- Front-channel logout URL:** A text input field containing the value 'https://example.com/logout'.
- Implicit grant and hybrid flows:** Two checkboxes are present: 'Access tokens (used for implicit flows)' and 'ID tokens (used for implicit and hybrid flows)'. Both checkboxes are checked.

At the bottom of the dialog, there are two buttons: 'Configure' (highlighted with a red box) and 'Cancel'.

- i. Add `https://token.botframework.com/.auth/web/redirect` as the **Redirect URIs**.
 - ii. Under Implicit grant and hybrid flows section, select the checkbox for **Access tokens** and **ID tokens**.
 - iii. Select **Configure**.
5. Create a client secret for the application:
- a. In the left navigation, select **Certificate and secrets**.
 - b. Under the **Client secrets** tab, select **+ New client secret**.
 - c. In the dialog that displays:

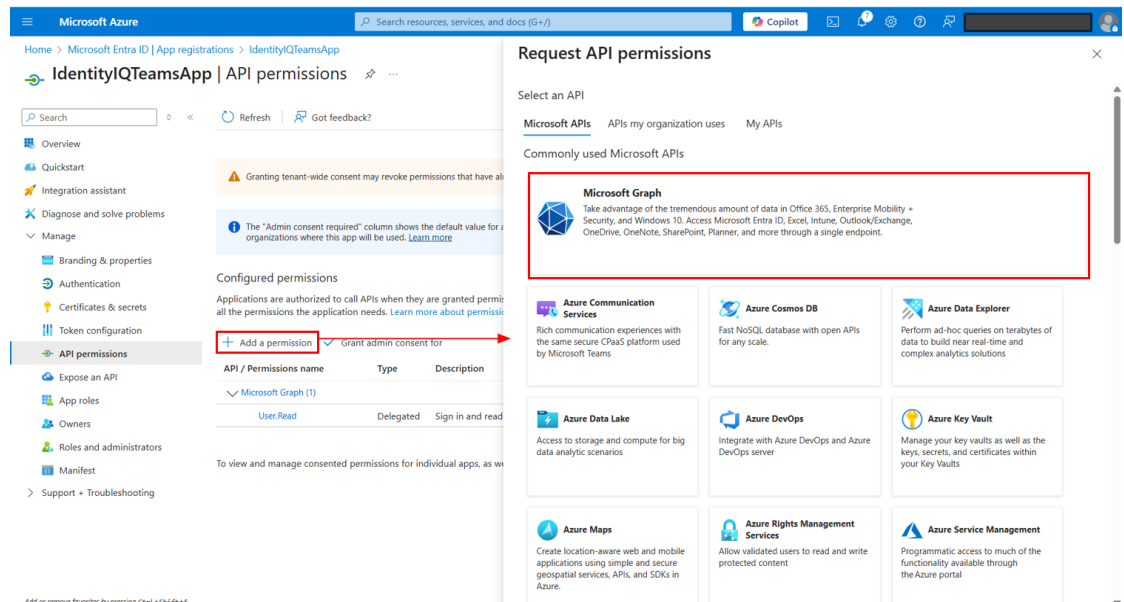


- i. Provide a **Description** for the secret.
- ii. Select a time of expiration in **Expires** field.
- iii. Select **Add**.

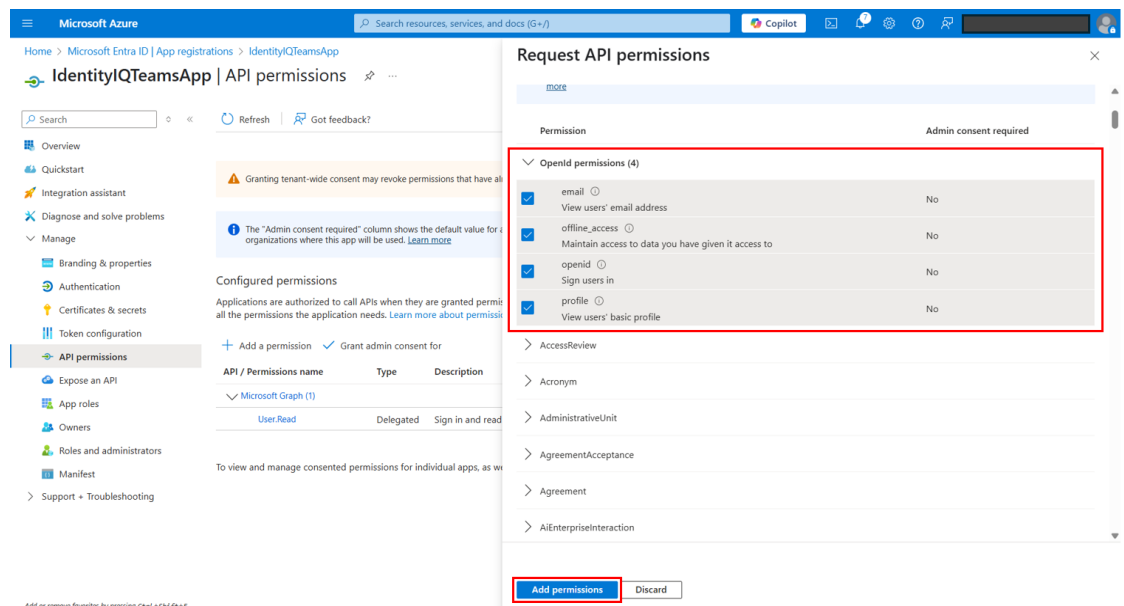
Note: After the client secrets are added, its details are displayed including description, expiration, value, and secret ID. These details will be required for the IdentityIQ service code configuration in [Creating an Azure Bot for IdentityIQ's Microsoft Teams](#) and [Installing and Configuring the IdentityIQ Service Code](#). Therefore, make a note of all the details and keep it safe.

6. Configure the API permissions for the application:
 - a. In the left navigation, select **API permissions**.
 - b. Select **+ Add a permission**.
 - c. In the dialog that displays:

i. Under the **Microsoft APIs** tab, select **Microsoft Graph**.



ii. Select **Delegated permissions**.



iii. Under **OpenId permissions**, add the following:

- email
- offline_access

- openid
- profile

iv. Select **Add permissions**

d. Select **+ Add a permission**.

e. In the dialog that displays, perform the following steps:

i. Under the My APIs tab, select API Access application created.

Note: A dialog appears and displays the **API Access Application** name, and the **Application ID URI** which was set in [Creating an API Access Application in Azure](#).

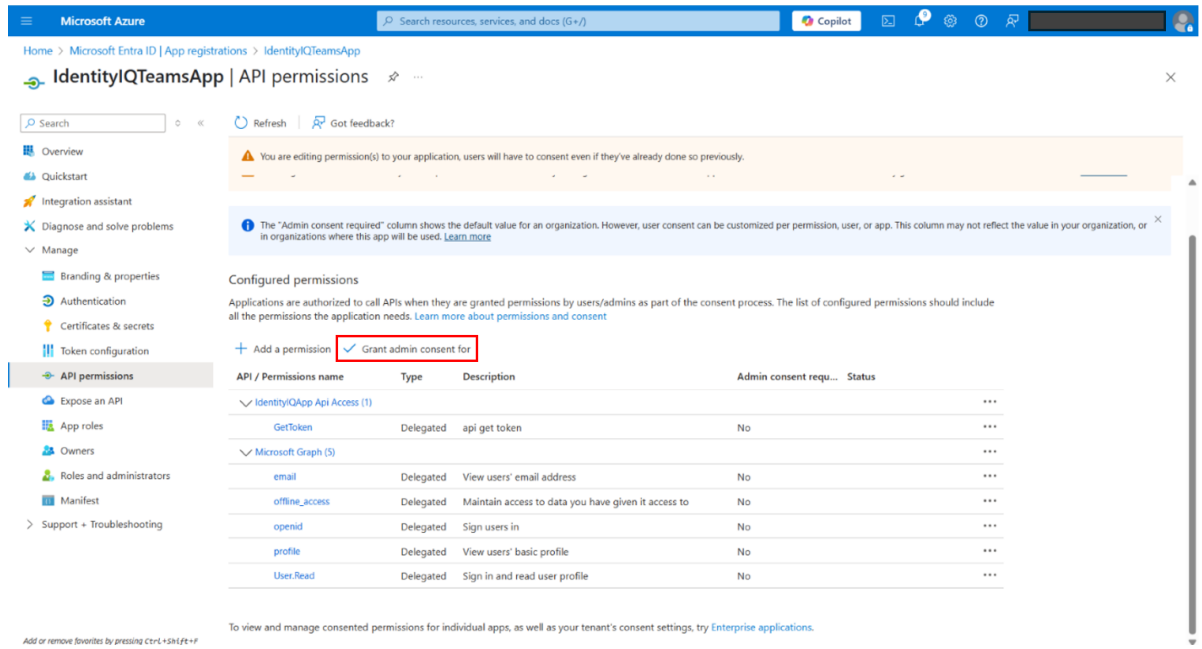
ii. Select **Delegated Permission**.

iii. Select **GetToken** checkbox.

Note: GetToken was created in [Creating an API Access Application in Azure](#).

iv. Select **Add permissions**.

f. On the API permissions page, select **Grant admin consent for <tenant name>**.



Microsoft Azure

Home > Microsoft Entra ID | App registrations > IdentityIQTeamsApp

IdentityIQTeamsApp | API permissions

Search resources, services, and docs (G+/I)

Refresh | Got feedback?

Overview

Quickstart

Integration assistant

Diagnose and solve problems

Manage

Branding & properties

Authentication

Certificates & secrets

Token configuration

API permissions

Expose an API

App roles

Owners

Roles and administrators

Manifest

Support + Troubleshooting

You are editing permission(s) to your application, users will have to consent even if they've already done so previously.

The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permission, user, or app. This column may not reflect the value in your organization, or in organizations where this app will be used. [Learn more](#)

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission ☒ Grant admin consent for

API / Permissions name	Type	Description	Admin consent requ...	Status
IdentityIQApp API Access (1)				...
GetToken	Delegated	api get token	No	...
Microsoft Graph (5)				...
email	Delegated	View users' email address	No	...
offline_access	Delegated	Maintain access to data you have given it access to	No	...
openid	Delegated	Sign users in	No	...
profile	Delegated	View users' basic profile	No	...
User.Read	Delegated	Sign in and read user profile	No	...

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

Add or remove favorites by pressing Ctrl+Shift+F

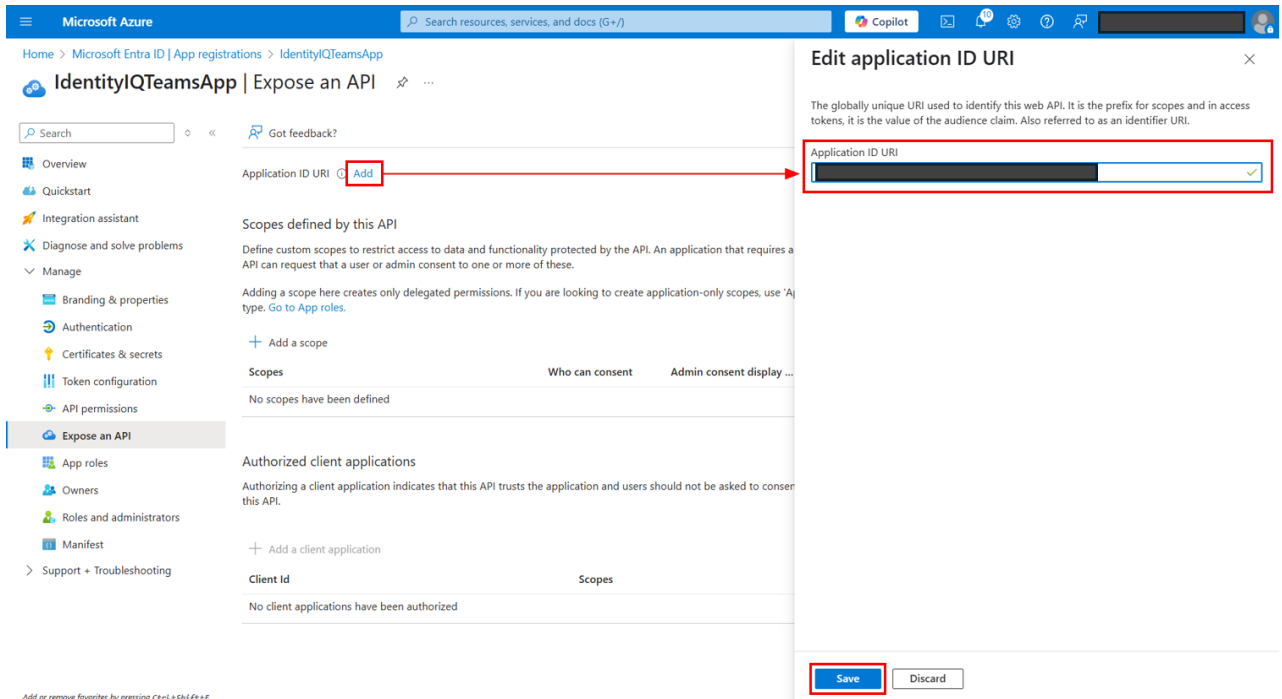
g. Select **Yes** on the confirmation pop-up. The page displays all the permissions that are now granted.

7. In the left navigation, select **Expose an API**.

8. Select **Add** for Application ID URI.

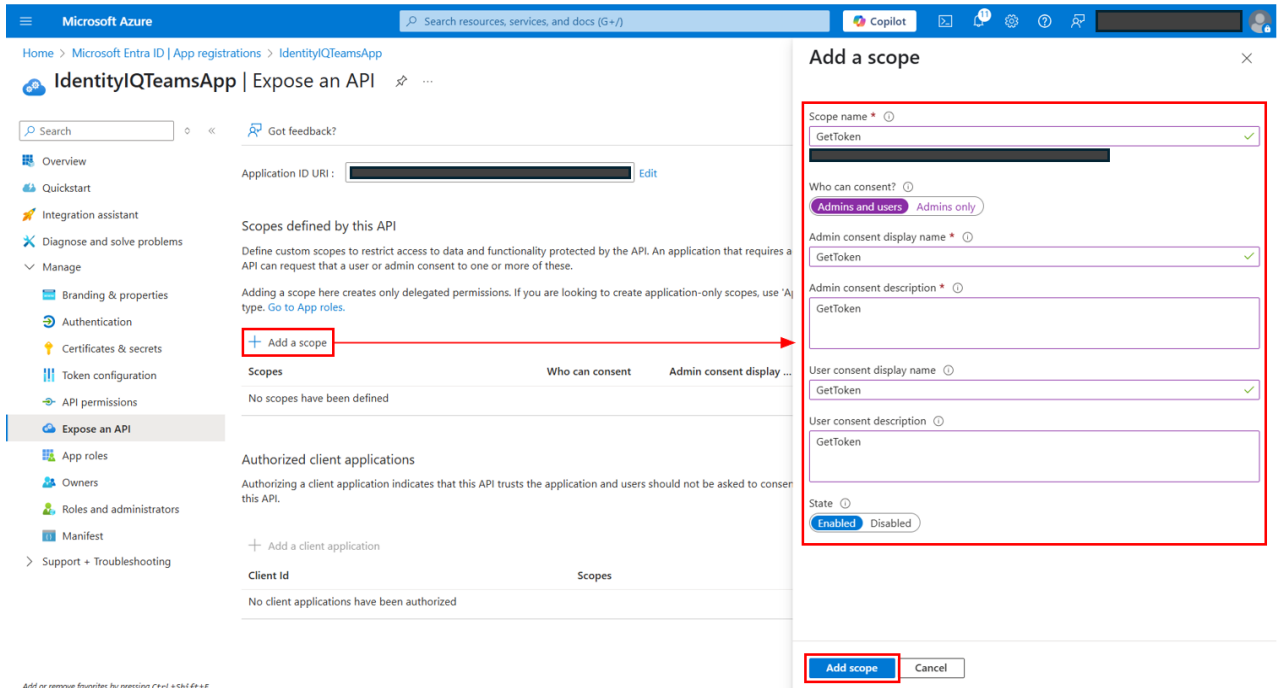
9. On the pop-up that displays, add `botid-` to the URI syntax.

For example, if the URI syntax is, `api://<application (client) id>`, update it to, `api://botid-<application (client) id>`.



10. Select **Save**.

11. Add a **GetToken** scope:



- a. On the Expose an API page, select **+ Add a scope**.
- b. In the dialog that displays:
 - i. Scope name – Type **GetToken**.
 - ii. Who can consent – Select **Admins and users**.
 - iii. Add display names and descriptions for **Admin consent** and **User consent**.
 - iv. Select **Enabled** state.
 - v. Select **Add scope**.

12. Add an `access_as_user` scope:

The screenshot shows the Microsoft Azure portal interface for the 'IdentityIQTeamsApp'. The 'Expose an API' section is active, and the 'Add a scope' dialog is open. The dialog contains the following fields and values:

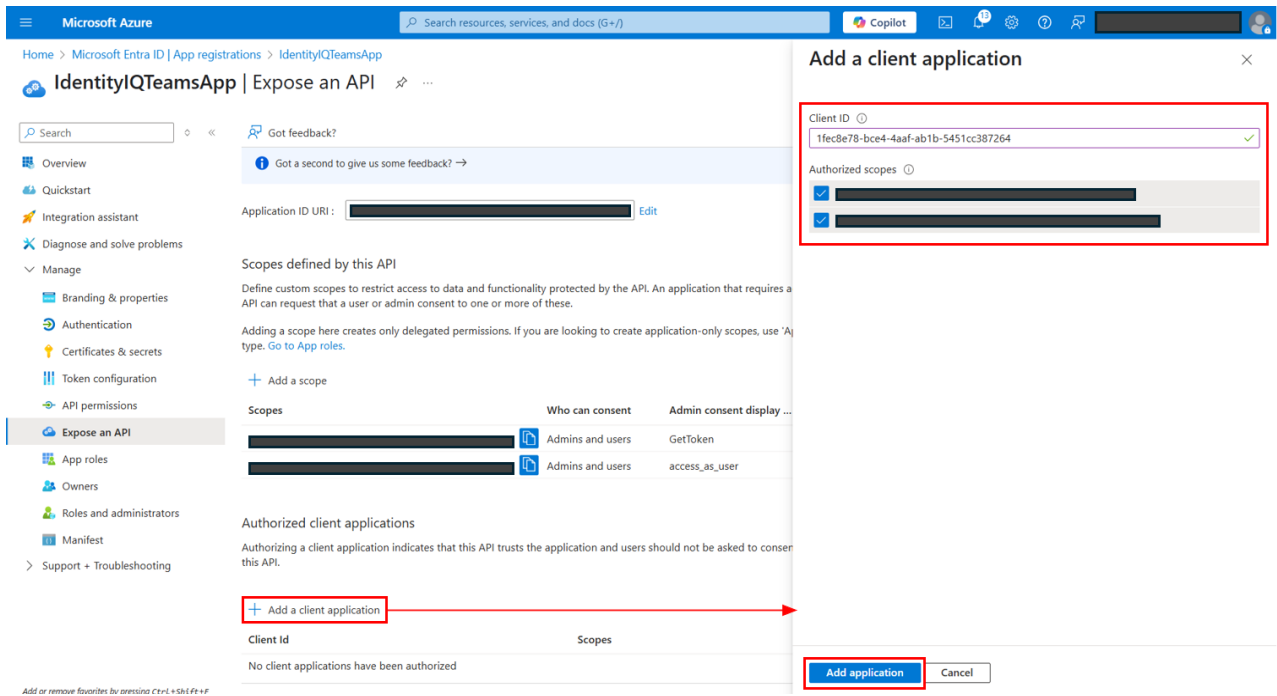
- Scope name:** `access_as_user` (with a green checkmark)
- Who can consent:** `Admins and users` (selected from a dropdown)
- Admin consent display name:** `access_as_user` (with a green checkmark)
- Admin consent description:** `access_as_user`
- User consent display name:** `access_as_user` (with a green checkmark)
- User consent description:** `access_as_user`
- State:** `Enabled` (selected from a dropdown)

The 'Add scope' button at the bottom of the dialog is highlighted with a red box. The background shows the 'Expose an API' page with a table of scopes and a list of authorized client applications.

- a. On the Expose an API page, select **+ Add a scope**.
- b. In the dialog that displays:
 - i. Scope name – Type `access_as_user`.
 - ii. Who can consent – Select **Admins and users**.

- iii. Add display names and descriptions for **Admin consent** and **User consent**.
- iv. Select **Enabled** state.
- v. Select **Add scope**.

13. Authorize desktop and mobile applications to use this application.



- a. On the Expose an API page, select **+ Add a client application**.
- b. In the dialog that displays:
 - i. Add 5e3ce6c0-2b1f-4285-8d4b-75ee78787346 into **Client ID** field.

Note: This is Global Client ID for Microsoft Teams web. This value is provided by Microsoft; refer to Microsoft's documentation for details about the Client ID.

- ii. Select the Authorized scopes for **GetToken** and **access_as_user**.
 - iii. Select **Add application**.
- c. On the Expose an API page, select **+ Add a client application**.

- d. In the dialog that displays:
 - i. Add 1fec8e78-bce4-4aaf-ab1b-5451cc387264 into **Client ID** field.
- Note: This is Global Client ID for Microsoft Teams mobile. This value is provided by Microsoft; refer to Microsoft's documentation for details about the Client ID.
- ii. Select the Authorized scopes for **GetToken** and **access_as_user**.
 - iii. Select **Add application**.

You now have successfully created a Microsoft Team application in Azure. For next step, refer [Creating an Azure Active Directory Application in IdentityIQ](#).

Creating an Azure Active Directory Application in IdentityIQ

To facilitate SAML-based Single sign-on (SSO) authentication in IdentityIQ, it is essential to establish an Azure Active Directory (Azure AD) application. This application serves as the Identity Provider (IdP) for the SAML authentication process.

To create the application in IdentityIQ, perform the following steps:

1. Log in as an IdentityIQ administrator and navigate to **Applications > Application Definition**.
2. Select **Add New Application**.
3. Under the **Details** tab, provide the following details:
 - a. **Name** of the application.
 - b. **Owner** of the application.
 - c. **Application Type** – Select **Azure Active Directory**. On selecting the application type, additional tabs will be displayed on the page.
4. Under the Configuration tab, provide the following details:
 - a. Add **Client ID** noted during [Creating a Connector Application in Azure](#).
 - b. Add **Client Secret Key** noted during [Creating a Connector Application in Azure](#).
 - c. Type a **Domain Name** of the format; <tenant name>.onmicrosoft.com.

- d. Select **Enable Microsoft Teams Notifications** checkbox. On selecting the checkbox, the following fields will be displayed:
 - Add **Microsoft Teams Bot URL** to notify endpoint. For example, `https://<VM Name/IP>:<PORT>/<TeamsAPPID without dash>/api/notify`.
 - Add **Tenant ID**. You can get the ID on the Application overview page on Azure Active Directory.
 - **Microsoft Teams Approvals Tab ID** is the Application (client) ID as mentioned in [Creating an SSO Entra Application Proxy in Azure](#).
 - **Microsoft Teams Approvals Bot ID** is the Application (client) ID as mentioned in [Creating a Microsoft Teams Application for IdentityIQ in Azure](#).
 - **Microsoft Teams Bot Secret** is the App (Client) Secret as mentioned in [Creating a Microsoft Teams Application for IdentityIQ in Azure](#).

5. Under the **Correlation** tab, provide the following details:

Note: Determine the best attribute to use based on the accounts created, or set it to what you want it to be and create accounts accordingly.

- a. Select **New**. The **Correlation Wizard** window is displayed.
- b. Select **Next**. The **Name Configuration** settings are displayed.
- c. Enter a name for this new configuration and select **Next**. The **Define Attribute based Correlation Assignments** settings are displayed.
- d. Select **ObjectId** as the Application Attribute.
- e. Select **Display Name** as the Identity Attribute.
- f. Select **Add**.
- g. Select **Save**.
- h. On the Edit Application <Name> page, under Account Correlation, select the newly created Configuration.
- i. Select **Save**.

6. Debug page must be updated with IPs and Entra URL for Approvals screen to load in IdentityIQ application in Microsoft Team. Perform the following steps to update IPs and Entra URL in debug page:
 - a. Navigate to the Debug page through the Debug URL: `https://<hostname>/identityiq/debug`.
 - b. Select **Application** object.
 - c. Select the Azure Active Directory Application created in IdentityIQ. The Object Editor will open, showing the configuration settings in XML format.
 - d. Add the following snippet the XML:

```
<Attributes>
<Map>
.....
<entry key="entraProxyConnectorIps">
  <value>
    <List>
      <String> <Connector Group VM IP/IP's> </String>
    </List>
  </value>
</entry>
</Map>
</Attributes>
```

Note: entraProxyConnectorIps contains a list of IPs or CIDR (Classless Inter-Domain Routing) block. Keep the connector IPs in this configuration. System compares these IPs with `HttpRequest.getRemoteAddr()` to validate if the request has originated from the configured IPs.

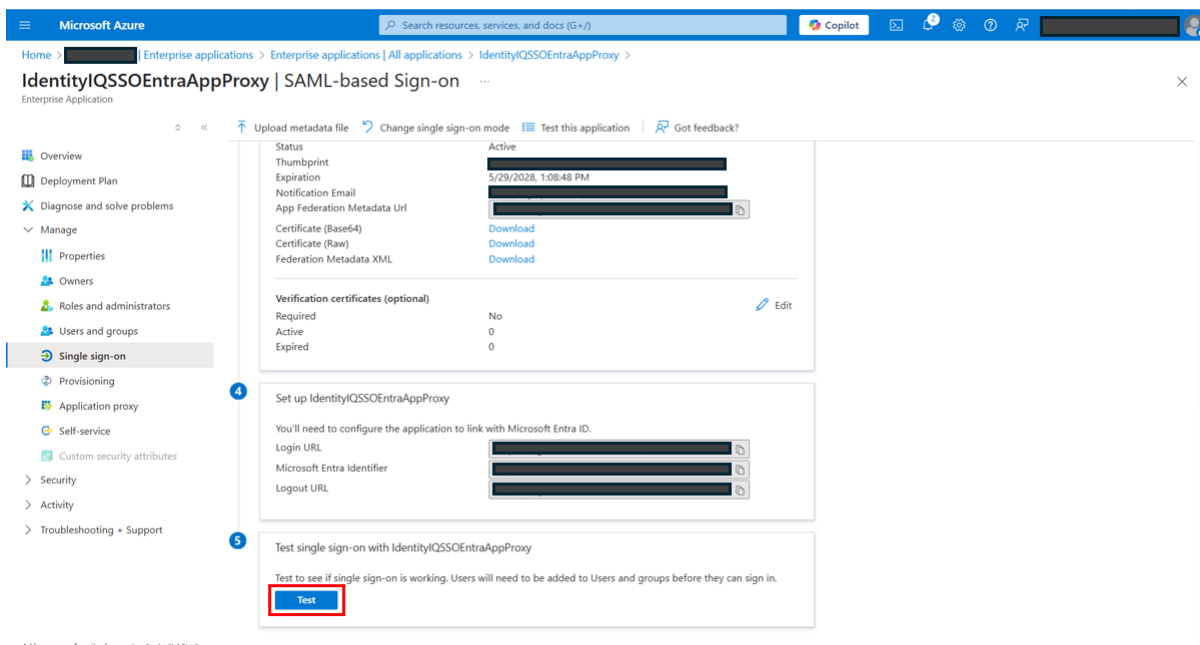
- e. Select **Save**
7. Navigate to **Setup > Tasks**.
8. Create an **Account Aggregation** task for the Azure Active Directory Application and run it. For more information on how to create and run a task in IdentityIQ, see the *SailPoint IdentityIQ Tasks Guide*.
9. Create an **Account Group Aggregation** task for the Azure Active Directory Application and run it. For more information on how to create and run a task in IdentityIQ, see the *SailPoint IdentityIQ Tasks Guide*.

Testing SSO Entra Application Proxy Connection

There are two methods to test the SSO Entra Application Proxy connection. You can use one of the two methods.

1. First method:

- a. Navigate to Azure portal and open the SSO Entra Application Proxy that was created in [Creating an SSO Entra Application Proxy in Azure](#).
- b. Under **Manage > Single sign-on**, select **SAML** tile.
- c. Select **Test**.



2. Second method:

- a. Open the SSO Entra Application Proxy URL in a browser.

If you're redirected to the Microsoft login page, it indicates that the SSO Entra Application Proxy connection test is successful.

You now have successfully created an Azure Active Directory application in IdentityIQ. For next step, refer [Configuring Single Sign-On to IdentityIQ from Microsoft Teams](#).

Configuring Single Sign-On to IdentityIQ from Microsoft Teams

IdentityIQ Microsoft Teams requires a SAML-based Single sign-on (SSO) configuration. Enabling SSO for Microsoft Teams and IdentityIQ is required because it streamlines the login process for Microsoft Teams users, allowing them to bypass the IdentityIQ login when they click IdentityIQ links in Microsoft Teams notification.

Enabling single sign-on from Microsoft Teams to IdentityIQ is a three-step process that requires creating an SSO Provider Application in Azure, configuring SAML SSO within Azure for the Provider Application, then using some of the parameters from the Azure steps to enable SAML SSO within IdentityIQ.

Create an SSO Provider Application in Azure Active Directory

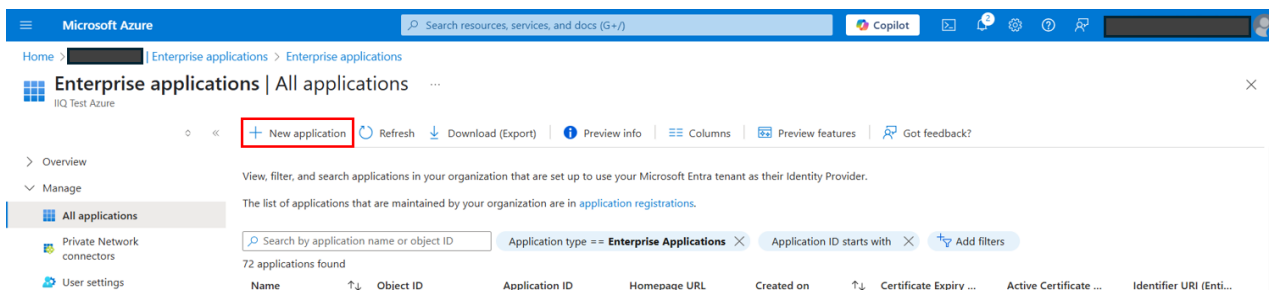
Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

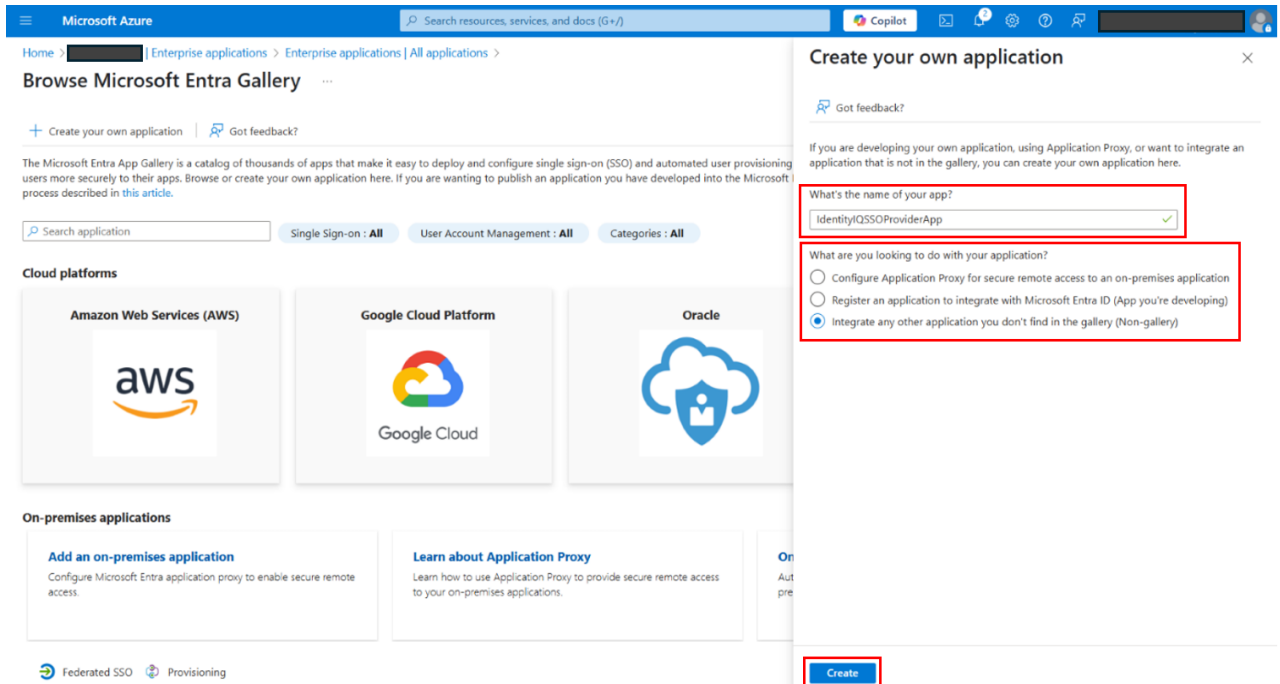
The Azure SSO Provider Application facilitates seamless authentication to IdentityIQ when users follow a link embedded in a Microsoft Teams notification.

Perform the following steps to create an SSO Provider Application:

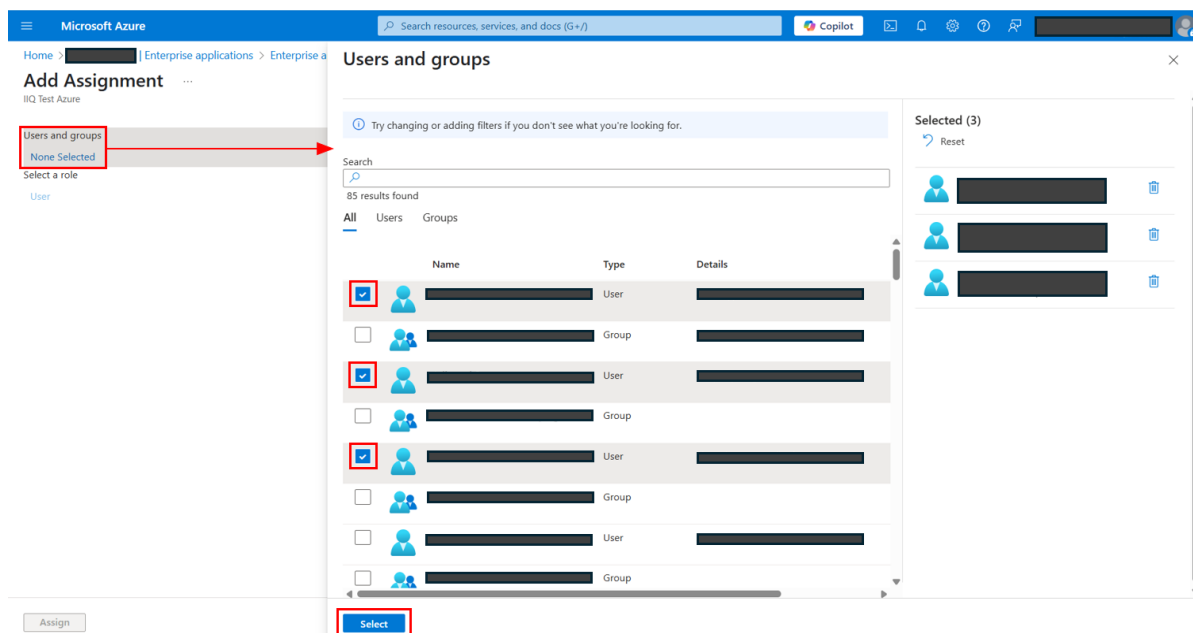
1. Sign into **Azure** portal and navigate to your **Microsoft Entra ID**.
2. Under **Manage > Enterprise applications**, select **+ New application**.



3. Select **+ Create your own application**.
4. On the Application creation page, provide the following details:



- a. Enter a **Name** for your application. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.
 - b. Select **Integrate any other application you don't find in the gallery**.
 - c. Select **Create**. Once the creation process is complete, Azure opens a Properties page for the application.
5. Define the users and groups that can use SAML SSO with IdentityIQ:
 - a. On Overview page, select **Assign users and groups** tile.
Alternatively, navigate to **Manage > Users and groups** and then select **+ Add user/group**.
 - b. On Add Assignment page, under **Users and groups**, select **None Selected**.



- c. On the Users and groups dialog, provide the following details:
 - i. Select the users and groups that you want to be able to use SAML SSO.
 - ii. Select the **Select** button.
 - d. Select **Assign**.
6. In the left navigation, select **Single sign-on**.
 7. Select the **SAML** tile.
 8. In the **Basic SAML Configuration** section, select **Edit**.
 9. On the Configuration dialog, provide the following details:

Basic SAML Configuration

Identifier (Entity ID) *

Reply URL (Assertion Consumer Service URL) *

Sign on URL (Optional)

Relay State (Optional)

Attributes & Claims

Name	Type	Required
givenname	string	Optional
surname	string	Optional
emailaddress	string	Optional
name	string	Optional
Unique User Identifier	string	Required

SAML Certificates

Name	Status	Thumbprint	Expiration	Notification Email
Token signing certificate	Active		2/24/2030	

- Add an **Identifier (Entity ID)**. This can be any name, but typically takes the form of a URL: `https://<IdentityIQ Server IP>:<Port>/<root context>`. This value is used to link Azure and IdentityIQ. Make a note of it to use later when configuring SSO within IdentityIQ.
- Add a **Reply URL (Assertion Consumer Service URL)** to your IdentityIQ instance, in this format: `https://<IdentityIQ Server IP>:<Port>/<root context>/home.jsf`. This URL must be accessible to your Microsoft Teams users on your network.
- Add a **Sign on URL** to your IdentityIQ instance, in this format: `https://<IdentityIQ Server IP>:<Port>/<root context>`.
- Do not add a **Logout URL**.
- Select **Save**.

- In the **Attributes & Claims** section, select **Edit**.
- On the Attributes & Claims page, select **Unique User Identifier (Name ID)**.
- On the Manage claim page, provide the following details:

The screenshot shows the 'Manage claim' page in the Microsoft Azure portal. The page has a blue header with the Microsoft Azure logo and a search bar. Below the header, there's a breadcrumb trail: Home > IdentityIQSSOProviderApp | SAML-based Sign-on > SAML-based Sign-on > Attributes & Claims >. The main title is 'Manage claim'. Below the title, there are three buttons: 'Save' (highlighted with a red box), 'Discard changes', and 'Got feedback?'. The form contains several fields: 'Name' with the value 'nameidentifier', 'Namespace' with the value 'http://schemas.xmlsoap.org/ws/2005/05/identity/claims', and a section titled 'Choose name identifier format'. This section has three fields: 'Name identifier format' with a dropdown menu set to 'Unspecified', 'Source' with radio buttons for 'Attribute' (selected), 'Transformation', and 'Directory schema extension', and 'Source attribute' with a dropdown menu set to 'user.objectid'. Below these fields, there are two expandable sections: 'Claim conditions' and 'Advanced SAML claims options'.

a. Select **Unspecified** for Name identifier format.

b. Select **user.objectId** for Source attribute.

c. Select **Save**.

13. In the **SAML Signing Certificate** section, download and save the **Certificate (Base64)**. You will need this later, when configuring SAML SSO within IdentityIQ.

14. In the **Set up (name)** section, make a note of the Login URL, Microsoft Entra Identifier, and Logout URL that are listed. You will need these later, when configuring SAML SSO within IdentityIQ.

Configuring Single Sign-On in IdentityIQ

Perform the following steps for enabling Single sign-on in IdentityIQ:

1. Select the **gear menu > Global Settings > Login Configuration**.
2. Select the **SSO Configuration** tab.
3. Check to select the **Enable SAML Based Single Sign-On (SSO)** box. This opens a panel for entering SAML Based SSO details.
4. In the **Identity Provider Settings** section:
 - a. Enter an **Entity ID / Issuer**. This must match the **Microsoft Entra Identifier** from the **Set up (name)** section in Azure. (See step 14 in the previous section.)
 - b. Enter a **SSO Login URL**. This must match the **Login URL** from the **Set up (name)** section in Azure. (See step 14 in the previous section.)

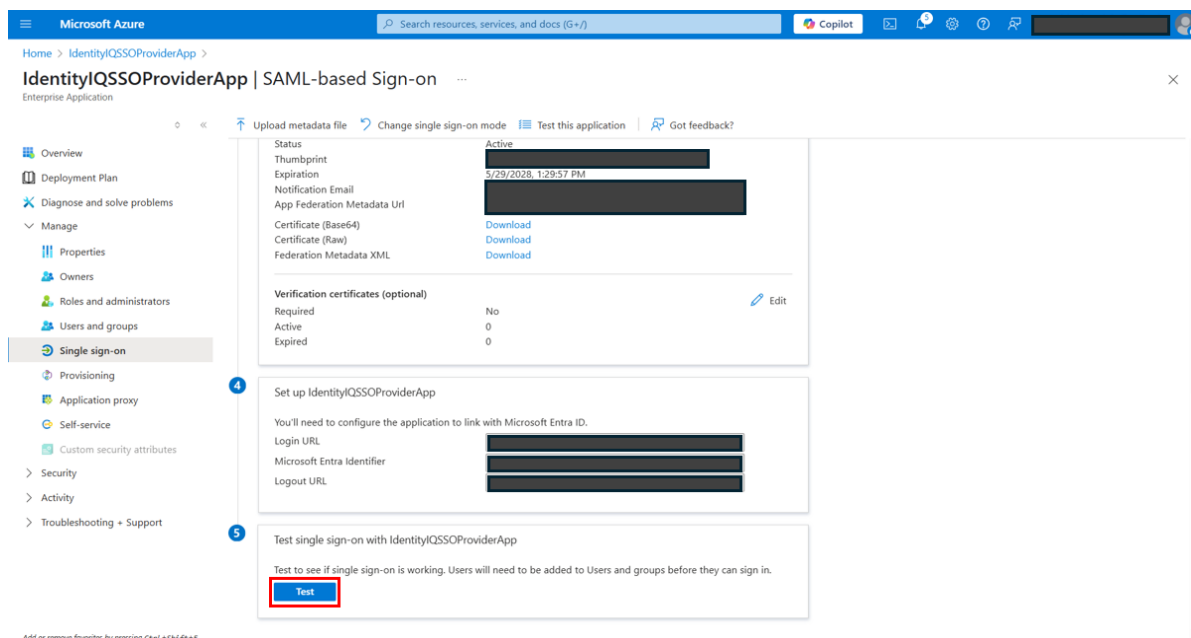
- c. Enter your **Public X.509 Certificate**. This is the certificate that was downloaded and saved in the Azure **SAML Signing Certificate** panel, as described the previous section.
5. In the **Service Provider (IdentityIQ) Settings** section:
 - a. Enter an **Entity ID / Issuer**. This must match the **Identifier (Entity ID)** value that was set in Azure in the **Basic SAML Configuration** panel, as described in the previous section.
 - b. Enter a **SAML URL**. This must match the **Reply URL (Assertion Consumer Service URL)** that was set Azure in the **Basic SAML Configuration** panel, as described in the previous section.
 - c. Set **SAML Binding** to HTTP POST.
 - d. Set the **SAML Name ID Format** to **unspecified**, if you are using the default identifiers as described in step 12 in the previous section. If you have opted to use a different set of values for uniquely identifying users, select the appropriate value here.
 - e. The **SAML Correlation Rule** manages the correlation of IdentityIQ identities to Azure/Teams identities. IdentityIQ provides a sample rule, **ExampleAzureActiveDirectoryUsingTenantIdSAML**. This rule is included in the `examplerules.xml` file, located in the `[install_directory]WEB-INF\config` directory. If you are using the recommended default identifiers, you can use the sample rule without modification. If you have opted to use different unique identifiers, you can either edit this rule, or use a different rule that you have developed.
6. **Save** your changes.

For more information on configuring your IdentityIQ instance for SSO authentication, see **SSO Configuration** section of the *SailPoint IdentityIQ System Configuration Guide*

Testing SSO Provider Application Connection

Use one of the two methods mentioned below to test the SSO Provider Application connection.

1. First method:
 - a. Navigate to Azure portal and open the SSO Provider Application.
 - b. Under **Manage > Single sign-on**, select **SAML** tile.

c. Select **Test**.

2. Second Method:

a. Open the SSO Provider Application URL in a browser.

If you're redirected to the Microsoft login page, it indicates that the SSO Provider Application connection test is successful.

You now have successfully created an SSO Provider Application in Azure. For next step, refer [Creating a Chat Application Proxy for IdentityIQ in Azure](#).

Creating a Chat Application Proxy for IdentityIQ in Azure

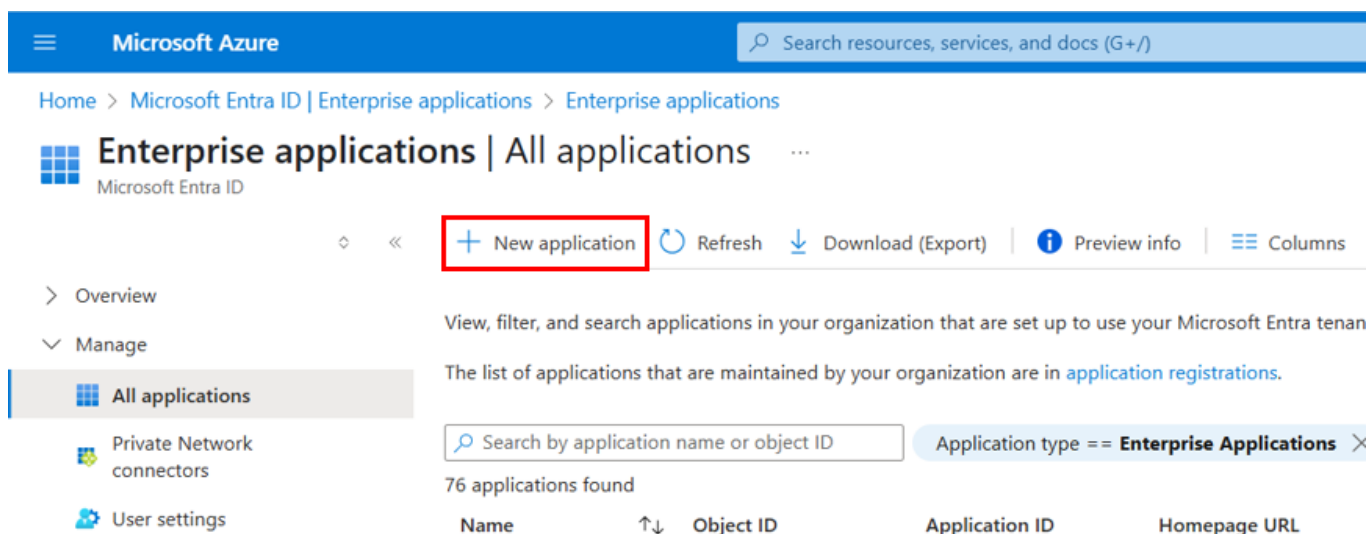
The Chat Proxy application acts as an Azure Entra Application Proxy, securing the Azure bot message endpoint (/api/messages) to facilitate communication between Microsoft Teams and IdentityIQ application running in your enterprise. By establishing an Entra proxy URL for the Microsoft Teams bot server endpoint, it ensures that messages sent from Microsoft Teams to the bot are securely routed. This setup leverages Azure Entra's capabilities to provide authenticated access, apply conditional access policies, and terminate traffic in the cloud, thereby enhancing security and compliance.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

Perform the following steps to create a Chat Application Proxy in Azure:

1. Sign into Azure portal and navigate to your **Microsoft Entra ID**.
2. Under **Manage > Enterprise applications**, select **+ New application**.



3. On the Microsoft Entra Gallery page, select **Add on-premises application**.
4. On the Add application page, provide the following details:

a. Under the **Basic** tab:

Microsoft Azure

Home > Microsoft Entra ID > Enterprise applications > Enterprise applications > All applications > Browse Microsoft Entra Gallery >

Add your own on-premises application

Got feedback?

Introducing Application segments for wild card applications - a simpler way to publish complex distributed applications made up of multiple regular apps and manage their CORS settings. [Learn more](#)

Basic Advanced

Maintenance mode ☐

Name * IdentityIQChatProxy

Internal Url *

External Url https://

Application segments Add application segments

Pre Authentication Microsoft Entra ID

Connector Group

SSL Certificate No SSL certificate required.

We recommend at least two active connectors in selected group 'test'. [Click here to download a connector or manage your connector groups.](#)

Create Discard

- i. **Maintenance mode** – Keep it unselected. This is the maintenance page that users see during maintenance and downtime events.
- ii. **Name** – Name of your Chat Application Proxy in Azure. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.
- iii. **Internal Url** – Add the **Internal Url** in the format; `https://<VM Name Bot Server URL>:<PORT>/<IdentityIQ Teams APP ID without Dashes>/api/`.
- iv. **External Url** – An auto-generated **external URL** with application name options as input. The URL can be customized.

Note: This URL is the bot server private bot endpoint within your enterprise that the public proxy will direct messages to.

Note: This is your bot server public endpoint which Microsoft Teams will communicate with for IdentityIQ chat. The bot messaging endpoint is securely exposed externally to Microsoft Teams via an Entra Id proxy. Only secured messages from Microsoft Teams can be successfully processed by the messaging endpoint.

- v. **Pre Authentication** – Attribute for pre authentication. By default, Microsoft Entra ID is selected. Select **Passthrough** from dropdown.
 - vi. **Connector Group** – Select the appropriate connector group from the dropdown.
- b. Under the **Advance** tab, select **Validate Backend SSL certificate** checkbox.
- Note: If your virtual machine (VM) does not have a valid certificate, unselect the **Validate Backend SSL certificate** checkbox.
- c. Select **Create**.

You now have successfully created a Chat Application Proxy in Azure. For next step, refer [Creating an Azure Bot for IdentityIQ's Microsoft Teams](#).

Creating an Azure Bot for IdentityIQ's Microsoft Teams

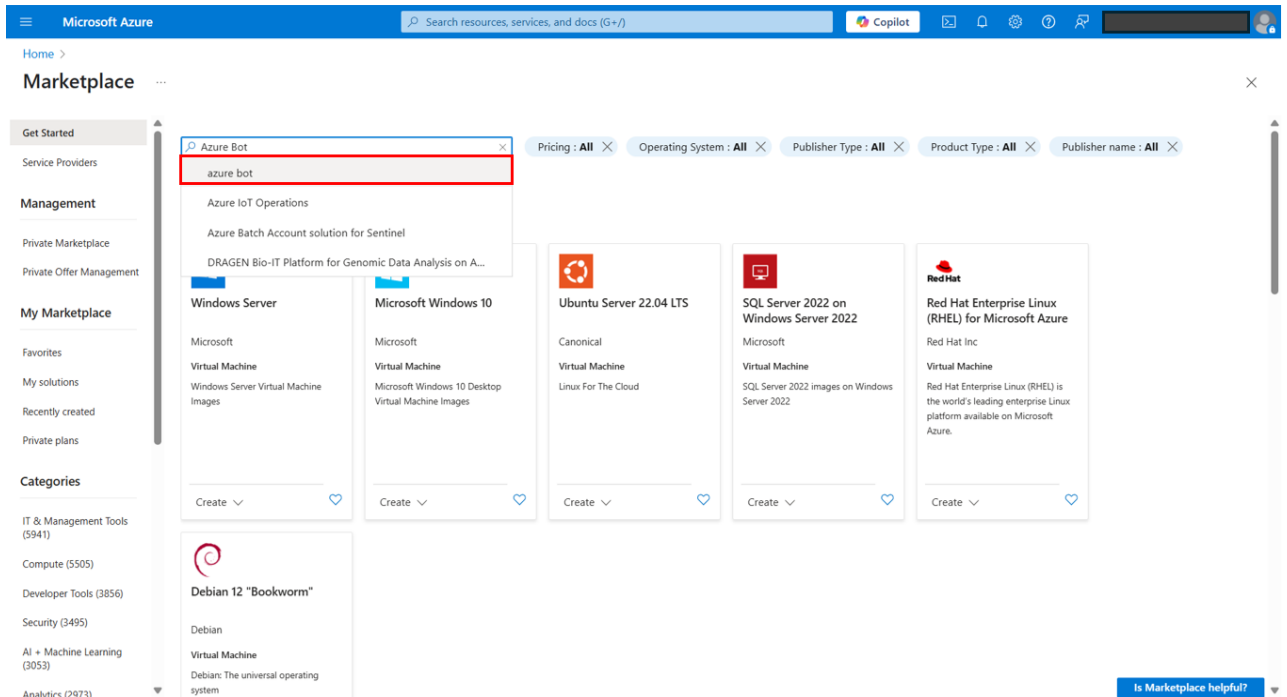
The Azure bot utilizes Microsoft's AI chatbot framework to facilitate messaging between Microsoft Teams and IdentityIQ. Additionally, it stores the configuration necessary to access the IdentityIQ service code. This setup enables seamless communication and integration between the two platforms.

Note: It is essential to follow the specified sequence of steps as mentioned in the document when creating the necessary applications for integrating IdentityIQ with Microsoft Teams. For a visual guide to the recommended setup order, refer to [Best Practices for Configuring IdentityIQ Microsoft Teams](#).

Important: This guide ONLY provides instructions for configuring specific Azure component configurations required to support IdentityIQ's Notifications and Access Request Approval work item features in Microsoft Teams. It is intended as an aid to implementers but should be used in conjunction with Microsoft's official documentation to ensure access to the most accurate and up-to-date information. For broader information on Azure or general setup tasks related to Microsoft Teams and SSO, please refer to [Microsoft's official documentation](#).

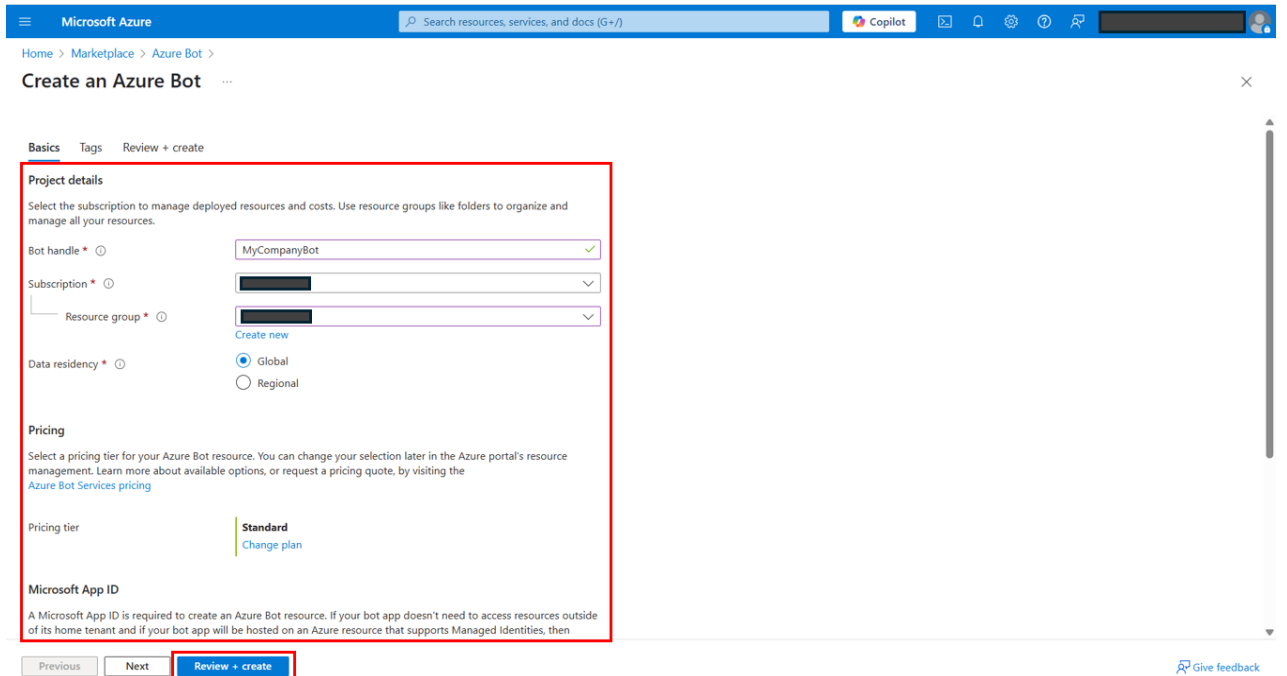
Perform the following steps to create Azure bot:

1. From your Azure home, use the search field to search for **Azure Bot**.
2. Select the result that appears in the **Marketplace** category.



3. Select **Create** for Azure Bot.

4. On the **Create an Azure Bot** page, provide the following details:



- a. Enter a **Bot handle** name.

Note: This is the name of your bot, that your users will see in the Microsoft Teams application. See [Best Practices for Configuring IdentityIQ Microsoft Teams](#) to define an appropriate application name.

- b. Select **Subscription** from drop down.
- c. Select your resource group for the bot.
- d. For Data Residency, select **Global**.
- e. Select your pricing plan.
- f. For **Type of app**, select the **Single Tenant** option.
- g. For **Creation type**, select **Use existing app registration**. This is how you link the bot to your Microsoft Teams application.
- h. Enter **App ID** from the Microsoft Teams application that was created in [Creating a Microsoft Teams Application for IdentityIQ in Azure](#).
- i. Select **Review + create** to create the bot and link it to your Microsoft Teams application.

5. On the review screen, select **Create** to confirm the bot creation.

6. When the deployment process is complete, configure the bot:

- a. Select **Go to Resource**.
- b. In the left navigation, select **Settings > Configuration**.
- c. Enter a **Messaging endpoint**. The Messaging endpoint for the bot service needs to be the external URL of the chat bot proxy. For example: `https://<iiqchatproxy_external_url>/<identityiq_teams_id_withoutdash>/api/message`.

Note: Messaging endpoint is the External Url of Chat Application Proxy. See [Creating a Chat Application Proxy for IdentityIQ in Azure](#) for details

- d. Select **Apply**.
- e. Select **Add OAuth Connection Settings** - this connection manages tokens for SSO authentication.

- i. Enter a **Name** for the new connection setting. Do not include spaces in the name.
- ii. For **Service provider**, select **Azure Active Directory v2**.
- iii. Enter the **Client ID** and **Client secret** from your Microsoft Teams application. See [Creating a Microsoft Teams Application for IdentityIQ in Azure](#) for details.
- iv. For **Token Exchange URL**, enter the **Application ID URI** that was created during [Creating an API Access Application in Azure](#).
- v. Enter your **Tenant ID**.
- vi. For **Scope**, enter the scope created for your API Access application. To find this value, navigate to your API Access application, select **Expose an API**, and copy the full scope from the **Scopes** field, to enter here. See [Creating an API Access Application in Azure](#) for details.
- vii. Select **Save**.
- viii. Select **Add an OAuth connection settings** button.
- ix. Select **Test Connection**.

Note: You can run a test to obtain a token, which can be useful, though it doesn't guarantee that everything is configured properly. However, if there is an issue, the test will fail.

- x. On the left navigation, select **Channels**.
- xi. Navigate to **Available channels**.
- xii. Select **Microsoft Teams**.
- xiii. Select and accept Terms of service.
- xiv. Select **Microsoft Teams Commercial**.
- xv. Select **Apply** to save all your changes.

You now have successfully created an Azure Bot. For next step, refer [Installing and Configuring the IdentityIQ Service Code](#).

Installing and Configuring the IdentityIQ Service Code

The IdentityIQ service code is provided by SailPoint. The service code is packaged into platform-specific zip files and can be downloaded from [Compass](#). Choose the zip file appropriate for your platform.

The Linux zip file includes an executable file, an `env.template` file, a shell script and service script to assist with installing the code as a service.

The Windows zip file contains an executable file and an `env.template` file.

Once you have downloaded the zip appropriate to your environment, follow the below steps:

1. Extract the zip to the private server dedicated to hosting the service code. See the section on [Prerequisites for Integrating Microsoft Teams with IdentityIQ](#) for more information.
2. The zip file includes a template for configuring the IdentityIQ application and Microsoft Teams environment, named `env.template`. Copy this file and name it `.env`.
3. Create a `cert` directory in the root location where your extracted service code and `.env` file are.

Note: Once the zip file has been extracted, add a `BOT_HOME` environment variable to the server, set to the directory where your executable file, environment file, and `cert` directory are located (see steps below for details on the `cert` directory).

4. Edit the `.env` file to set configuration values for your environment. The values you *must* configure are listed below; some other values can be modified as needed for your environment. The variables in the file are commented, to give you information on their usage.
 - `LOG_FILENAME=identityiqbot.log`: the bot log file name. It provides a file path that makes the file easy to access.
 - `LOG_LEVEL=warn`: the bot log level. The valid values are `error`, `warn`, `info`, `debug`, and `trace`. For testing, it's a good idea to use `debug` or `trace` mode. `Trace` is especially useful for troubleshooting when issues come up.
 - `LOG_USE_LOCAL_TIMEZONE=true`: the bot time format. Set to `true` if the time zone of the local server should be used. Anything other than `true` is `false`.
 - `LOG_USE_JSON_STYLE=false`: specifies the style of log record. `True` writes a log record as a JSON object. Anything other than `true` writes a string.

- `DEFAULT_LANGUAGE=en-us`: this is used for looking up error message text.
- `NOTIFICATION_TEXT_FORMAT=markdown`: used to format the notification messages from IdentityIQ. Valid values are `xml`, `markdown`, or `plain`, and the variable defaults to `markdown`. This format only applies to notification messages based on `EmailTemplate`, but does not affect messages based on `AdaptiveCardNotificationTemplate`.
- `PUBLIC_HOSTNAME=<public DNS hostname>`: the hostname where you host the IdentityIQ bot service code. This must be a public DNS-resolvable hostname that resolves to the private IP of the server that will run the bot service code. This should be same as the hostname used for the Internal URL used in the chat application proxy. The traffic can be limited to a specific port, which is customizable. See [Pre-requisites for Integrating Microsoft Teams with IdentityIQ](#).
- `LOCAL_HOSTNAME=<local private IP address>`: use same value as for `PUBLIC_HOSTNAME`.
- `PUBLIC_PORT` and `PRIVATE_PORT`: the hosting ports for the IdentityIQ service code. The template provides default values which can be replaced, if necessary, with values specific to your installation. See the **Connectivity and Security Prerequisites** section in [Prerequisites for Integrating Microsoft Teams with IdentityIQ](#).
- `TENANT_ID=<Azure tenant ID>`: your Azure tenant ID.
- `PACKAGE_NAME=com.sailpoint.identityiqbot`: package name of the Microsoft Teams application. Ensure that the existing value is not changed.
- `WRAPPER_URL= https://microsoft-teams-bridge.sailpoint.com`: wrapper Application URL which handles the authentication for IdentityIQ Approval screen.
- `IIQ_UI_EXTERNAL_URL= <IIQAppSSOEntraProxy External URL (section 1)>`: URL for Microsoft Teams Entra Application Proxy.
- `IIQ_UI_EXTERNAL_APP_ID= <IIQAppSSOEntraProxy APP ID (section 1)>`: Application ID for Microsoft Teams Entra Application Proxy.
- `APPROVAL_TAB_NAME=My Approvals`: tab name on the Microsoft Teams app where requests are approved. The default value is `My Approvals`, which can be changed to some other suitable name.
- `APP_ID=<teams app ID>`: the Application (client) ID value that was set as part of configuring the Microsoft Teams application. You can find this value on the Overview page of the Microsoft Teams application. See [Creating a Microsoft Teams Application for IdentityIQ in Azure](#)

- APP_NAME=<teams app name>: the name of your Microsoft Teams app. See [Creating a Microsoft Teams Application for IdentityIQ in Azure](#)
- APP_SECRET=<encrypted teams app secret>: **IMPORTANT**: this value is required but will be set later, in a secondary step. See step 5 below for more information.
- SSO_CONNECTION_NAME=<SSO connection associated with azure bot>: this value was set in the **Add OAuth Connection Settings** when you configured the Azure bot. See [Creating an Azure Bot for IdentityIQ's Microsoft Teams](#)

Note: The name should not have any spaces.

- ALLOW_USER_TO_DISABLE_NOTIFICATIONS=false: notifications from IdentityIQ can be disabled; however, users will not be allowed to use the command.
 - ENCRYPTION_SECRET=<provide strong value>: provide an 8-character encryption secret. This can be any value you like, but if this value changes, any items encrypted must be re-encrypted.
 - IIQ_URL: the full URL to your installation of IdentityIQ. Use this format:
https://<host/ip>:<port>/<identityiq_home where <identityiq_home> is the directory in which you extracted the identityiq.war file during the IdentityIQ installation procedure. If you are using a load balancer to manage multiple IdentityIQ hosts, put the load balancer URL here. This URL must be accessible to the bot service.
 - SERVICE_URL=https://smba.trafficmanager.net/amer/: service URL for proactive messaging. Ensure that the URL is not changed.
 - AUTHORITY_HOST=https://login.microsoftonline.com: default setting for knowing if it is Microsoft application or not. Ensure that the URL is not changed.
 - JWK_PROVIDER=https://login.botframework.com/v1/.well-known/keys: used to validate signature of Microsoft JWTs. Ensure that the URL is not changed.
 - USER_AGENT=Microsoft-SkypeBotApi (Microsoft-BotFramework/3.0): value found in headers from Microsoft call to /api/messages endpoint - used for JWT verification. Ensure that the URL is not changed.
5. Encrypt and add your APP_SECRET. For your convenience, the IdentityIQ service code provides an encryption endpoint.
- a. Make sure rest of the .env file has been configured and saved before encrypting and adding the app secret.

- b. Start the bot (for example, by running the `identityiqbot.sh` script). Note that because the secret has not yet been encrypted, **you may see an error when initially starting the bot**. You can continue past this error to complete the process for encrypting the secret.
- c. Use the following private POST endpoint to encrypt messages such as app secret. Provide a JSON object as the payload and a JSON object will be returned with the encrypted value.

```
https://<private IP>:<private port>/util/encrypt
```

```
Input: {"message" : "some message to be encrypted"}
```

```
Output: { "status" : "success", "encrypted_message" : "....." }
```

An example curl command to encrypt the APP_SECRET is:

```
curl --insecure --request POST -H "Content-Type: application/json" --data
@secretSource.json "https://<local private ip>:<private port>/util/en-
crypt"
```

- d. Copy the returned value into the APP_SECRET=<encrypted teams app secret> field of the .env file.

6. Encrypt the CERT.

- a. Make a backup copy of the raw CERT.
- b. Convert the CERT to single line but preserve line endings by converting line ending characters to \n.

```
-----BEGIN CERTIFICATE-----\nMIIEATCCAumgAwIBAgIUSeQQ2/HwVWRtEbEa0ys2CqLhvSswDQYJKoZIhvcNAQEL\nBQAwgagxCzAJB
gNVBAYTAk1OMQswCQYDVQQIDAJUWDEPMA0GA1UEBwwGVGV4YWFz\nnMRIwEAYDVQQKDA1zYW1scG9pbmQxETAPBgNVBAsMCG1hZ25vbG1hMSE
wHwYDVQQLNDhhhdXdzZS50ZXN0LnNhaWxwb2ludC5jb20xMTAvBgkqhkiG9w0BCQEWImF1d3N1\nnLnRlc3Quc2FpbHBvaW50LmNvbUBnbWV
pbC5jb20wHhcNMjUwNTE0MTQyNjUxWWhcN\nnMjYwNTE0MTQyNjUxWjCBQDELMAKGA1UEBhMCSU4xCzAJBgNVBAGMA1RYMQ8wDQYD\nnVQQHDAZ
UZXhhYXN0EjAQBgNVBAoMCXNhaWxwb2ludDERMA8GA1UECwwIbWFnbm9s\nnaWExITAFBgNVBAMMGGF1d3N1LnRlc3Quc2FpbHBvaW50LmNvb
TExMC8GCSqGSIb3\nnDQEJARYiYXV3c2UudGVzdC5zYW1scG9pbmQxY29tQGdtYWIzLmNvbTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCA
QoCggEBAKYP1SmEJF2ffQwa1laUEuhhtSSGxyD\nn00quPS1wshEvL1DncmV5/V/+lyrC0Ey24t1nBXQGCfzdSGRoQmjQSuf5s2tZe2tw\nn0
nb1NoULSE1/vdjMLt2XTN5vu2bAcJS44iYIftsw0PAA5bG5zU6xY0vC1px0YDTe\nnvujAAXFdDGv0JNSMqFHIbg81t50dFxq2hI0LJ5Ks3z
1yHsyGsYpWLKZnLe6MHV\nnLI2C0mF5PYwPADN0t0PAQnBEReA3JCDpSQ0VeeDLVErkYWTkLQ8LnuwE3j1U4Z5g\nn9vLhp/W4qpz6TXr5Ja1
JML3fY0LEFPho/PbHVmpdg9M0qUJsq1YD0WECaWEEAAmH\nnMB8wHQYDVR00BBYEFB2Ly0yuzPqf8VVCLBDFG7nAKLdeMA0GCSqGSIb3DQEB
C
wUA\nA4IBABiV/95BqUgw12PM/TGkGPReptTry6DodUPbUYwXvCeBL0gZg3N1m2/s18D\nnvHECsZ1UZd0kxcm/MWUIbij1kmS0SKTTVyiV5
```

- c. Use the following private POST endpoint to encrypt CERT. Provide a JSON object as the payload and a JSON object will be returned with the encrypted value.

```
https://<private IP>:<private port>/util/encrypt
```

```
Example Input:{"message" : "-----BEGIN CERTIFICATE-----\nMIIEAT...\n-----
END CERTIFICATE-----\n":}
```

```
Output: { "status" : "success", "encrypted_message" : "....." }
```

An example curl command to encrypt the CERT is:

```
curl --insecure --request POST -H "Content-Type: application/json" --data
@botCertSource.json "https://<local private ip>:<private port>/util/en-
crypt" > botCertEncrypted.json
```

- d. Edit the file output to remove surrounding JSON formatting.

Note: Keep the encrypted_message property value only and remove the remaining properties and JSON format.

For Linux, use the command: `cat -vet <filename>`. If the output shows \$ at the end, remove it using the command: `truncate -s -1 <file name>`.

- e. Rename the edited file as bot.cert and save.
- f. Copy the encrypted file to cert directory. Make sure the cert directory and bot.cert file are set with rw permission for owner only.

7. Encrypt the key.

- a. Make a backup copy of the raw key.
- b. Convert the key to single line but preserve line endings by converting line ending characters to \n.

```
-----BEGIN PRIVATE KEY-----\nMIIEVAIBADANBgkqhkiG9w0BAQEFAASCCKYwggSiAgEAAoIBAQCmD9UphCRdn30M\nGtZW1BLoYbUkhssegxtKrj0tL
IRLy9Q53J1ef1f/pqwtBMtuLdZwV0Bgn83Uhk\naEJo0ErhebNrwXtrcN329TaFC0hNf73YzC7d10zeb7tmwHCUu0ImCH7bMKDwAOWx\nnuc10sWDrwtacdGA
03r7owAFxXQxr7ziTUjKhRyG4PjbeUHRcatoSNCyeSrN89ch7\nMhrGKViyMzy3ujB1fyyNgjpheT2MDwAzdLTjwEJwREXgNyQg6UkNFXngy7xK5GfK\nnyi0P
C57sBN45V0GeYPby4af1uKqc+k16+SWtSTC932NCx8T4aPz2x1ZqXYPTNK1C\nnbKpWA21hAgMBAAECggEAAAtVdDvu4ue1tokpoZ2A+9W1fy0iIjJxLaNGAvcs
LxbPB\nn1tLssTQZwn/rnDE1G0Qfdy21dkcsgEDviAwfxKAijNF7apshXFfEgXrH1bvIDjXX\nnNeVZdm5E1JogIn5okZ925j3VVZ7XoK6ZGpiFGwy7jrpFLcXD
ghnXCGdP6JfrjT0W\nn1KUMe9F0WzGBgM4oqHGY10X3nDL4aGNU7zhgcfLqxYj7LV7f1o7N1DNjvpOU+Dfa\nneC5HdjUnJFq87p7NavOmS2byCnqKSIqG+N65G
0oBeMtJ1RtqsAsZgKx1VonrEN+c\nnlzUfaAuEpg5N173vtjSZ5aTB70HWNijOMF1vVyphawK8gQDTZvz0Txi09uWsmLVW\nnKs0/aJXq2Pq4x3Ux5iueEhH05y
vLF4Uy+1CuFutq2joN+L+EBg9f4Syg02zDZGpM\nns9FQ86aY9aM4/xKLyKuM2S+rwSN22oeaV52ggACaM/b8ze0vd81gbg0/F4s7YUKM\nnywO6ToV6Un0ecJIG
DFY/Vgic6PwKBgQDjGC8ausWm07CT0xuTD0Mo1yuK6DqmH71H\nnmxEgWJo5n3yuaJ8/yyjJkhj1Lgc2ZprIR1BdGIh4cNCh4VvtRHiWqtzsHj05s/s\nncVNB
```

Use the following private POST endpoint to encrypt key. Provide a JSON object as the payload and a JSON object will be returned with the encrypted value.

```
https://<private IP>:<private port>/util/encrypt
```

Example Input: { "message" : "-----BEGIN KEY-----\nMIIEAT...\n-----END KEY---\n" : }

Output: { "status" : "success", "encrypted_message" : "..." }

An example Curl command to encrypt the key is:

```
curl --insecure --request POST -H "Content-Type: application/json" --data
@botkeySource.json "https://<local private ip>:<private port>/util/en-
crypt" > botKeyEncrypted.json
```


- c. Edit the file output to remove surrounding JSON formatting.

Note: Keep the `encrypted_message` property value only and remove the remaining properties and JSON format.

For Linux, use the command: `cat -vet <filename>`. If the output shows `$` at the end, remove it using the command: `truncate -s -1 <file name>`.

- d. Rename the edited file as `bot.key` and save.
- e. Copy the encrypted file to `cert` directory. Make sure the `cert` directory and `bot.key` file are set with `rw` permission for owner only.

8. Restart the bot server.

Allowing Microsoft Teams Users to Disable Notifications

The `ALLOW_USER_TO_DISABLE_NOTIFICATIONS` property in the `.env` file enables Microsoft Teams users to opt out of notifications, by entering the command `nonotifications` command directly within the Microsoft Teams environment. This option is disabled by default; if you want to allow users to disable notifications, set this value to `true`.

You now have successfully installed and configured the IdentityIQ service code. For next step, refer [Creating a Microsoft Teams Manifest](#).

Creating a Microsoft Teams Manifest

Once the `.env` file is completely configured, a bot manifest zip file can be created. This is the file users will use to install the bot into their local Microsoft Teams environment.

To create a manifest zip file, use the private `POST` endpoint, which is provided with the service code. This endpoint will require information from the `.env` file as input. When executed, the endpoint will generate a manifest zip file in the `<BOT_HOME>` location.

The response also includes the contents of the `manifest.json` file (which is packaged within the manifest zip file).

```
https://<private IP>:<private port>/util/manifest
```

```
Input: {"tenant_id" : "xxxxxxxx", "app_id" : "yyyyyyyy"}
```

```
Output: {"status" : "success", "manifest" : {.....}}
```

An example curl to generate the manifest zip file is:

```
curl -k -H "Content-type: application/json" -X POST -d '{"tenant_id" : "582b236a-807d-4fe6-a25b-000000000000", "app_id" : "60c8e70e-1411-4b7a-89f4-000000000000"}' https://bothost.acme.com:3988/util/manifest
```

After a successful post to the manifest endpoint, a zip file named `identityiqbot-<version>-manifest.zip` is created in `<BOT_HOME>`.

The manifest zip file contains the `manifest.json` file, as well as some configuration options such as SailPoint branding colors and outline logos. This can be customized by the users as necessary.

Once the manifest zip file is generated, it doesn't need to be generated again unless there are changes to the bot configuration in the `.env` file; for example, due to Azure portal changes.

If you face any challenge while generating a manifest zip file, verify the `tenant_id` and `app_id` provided are correct.

For more information on the service code, see [Installing and Configuring the IdentityIQ Service Code](#).

You now have successfully created a Microsoft Teams manifest. For next step, refer [Configuring API Authentication for Microsoft Teams in IdentityIQ](#).

Configuring API Authentication for Microsoft Teams in IdentityIQ

To enable token-based authentication between Microsoft Teams and IdentityIQ, you have already created an API Access application in Azure, and you will now use some of the values from the API Access application within IdentityIQ, to enable authentication. For related information about what to configure in Azure for token-based authentication, see [Creating an API Access Application in Azure](#).

To configure API Authentication in IdentityIQ:

1. Select **gear > Global Settings > API Authentication**.
2. On the **General Settings** tab, set an expiration value for tokens, in seconds. This value applies to both **Token Settings** (see below) and **OAuth Client Management** (see **API Authentication** in the **System Configuration** documentation).
3. Select the **Token Settings** tab, and provide the following values:
 - **Access Token Authentication Scope** – expected scope of the API access token issuer. This is the value you entered for **Scope Name** in [Creating an API Access Application in Azure](#); for example, `GetToken`.
 - **Access Token Authentication Audience** – suffix that identifies the service or system to which the call is directed. This is the value you appended to the **Application ID URI** in [Creating an API Access](#)

[Application in Azure](#) section; for example `api://<api access application>/identityiq/api`. The validator will ensure the SSO audience claim ends with this value.

- **Access Token Authentication Issuers** – identification of the SSO token provider. This field supports variable interpolation. Field values must be claims in the SSO token.
For example, `https://sts.windows.net/{{tid}}/`.
- **Correlation Variable** – the SSO claim used match the requesting user with an existing IdentityIQ user; the IdentityIQ default is `oid`.

4. **Save** your changes.

For more information on configuring your IdentityIQ instance for API authentication, refer to the **API Authentication** section of the *SailPoint IdentityIQ System Configuration Guide*.

You now have successfully configured the API authentication for Microsoft Teams. For next step, refer [Enabling Microsoft Teams Notifications in IdentityIQ](#).

Enabling Microsoft Teams Notifications in IdentityIQ

To enable IdentityIQ notifications to be sent on Microsoft Teams, you will need to select the option on the Global settings page in IdentityIQ.

Configuring Adaptive Card Template

1. Select the **gear** icon and navigate to **Global Settings > IdentityIQ Configuration > Notifications Settings**.
2. Select **Enable Microsoft Teams Notifications**. This will enable the users to receive IdentityIQ notifications on Microsoft Teams.
3. Select the appropriate templates for Adaptive card notifications for different events from the respective drop-down and accordingly it will be sent on Microsoft Teams. **Microsoft Teams Adaptive Card Templates** determine the contents of the notifications that are sent in Microsoft Teams. The screen displays four types of the adaptive card templates for different events:
 - a. For access request approval work item notices – sends adaptive card to the approver on Microsoft Teams when a new Access Request Approval work item is created in IdentityIQ.
 - b. For access request approval work item forwarding notices – sends adaptive card to the approver on Microsoft Teams when the Access Request Approval work item is forwarded to the approver from within IdentityIQ or Microsoft Teams.

- c. For access request approval work item assignment notices – sends adaptive card to the approver on Microsoft Teams when an Access Request Approval work item is assigned to the user from within IdentityIQ or Microsoft Teams.
- d. For access request approval work item assignment removal notices – sends adaptive card to the approver on Microsoft Teams when an Access Request Approval work item is unassigned from within IdentityIQ or Microsoft Teams.

Note: The adaptive card templates can be customized as necessary.

Editing Adaptive Card Template Content in Debug Page

Users can set notification messages sent on IdentityIQ application in Microsoft Teams for Access Request Approvals by editing Adaptive Card Template content in debug page.

To change the template content, perform the following steps:

1. Log in as an IdentityIQ administrator.
2. Navigate to the Debug pages through the Debug URL: `https://<hostname>/identityiq/debug`.
3. From the **Configuration Objects** dropdown list, select **AdaptiveCardNotificationTemplate**. A list of available templates displays.
4. Select the template name you want to change. An XML editor opens.
5. Change the template to render the notification message you want to set for that template/event.

Follow the velocity template inside the `<Body>` tag of the XML to generate JSON object that represent the adaptive card.

Configuring Email Template Notifications

If Adaptive card template is not configured, IdentityIQ sends notifications to Microsoft Teams users as per the **Notification Template** set in **Global Settings > IdentityIQ Configuration > Notifications Settings**.

You can select only one notification template for each type of notification (for example, **For reminder notices**, **For escalation notices**, etc.).

For information on how to edit Notification Templates, see the *SailPoint IdentityIQ System Configuration Guide*.

Formatting Options for Microsoft Teams Notifications

There is a property in the `.env` file that identifies the format of notification messages: `NOTIFICATION_TEXT_FORMAT`. The valid values are `xml`, `markdown`, and `plain`.

If the property has not been set in the `.env` file, or is not a valid value, it will default to markdown. For more details on how the text format is used by Teams, refer to [Microsoft's documentation](#) on formatting bot messages.

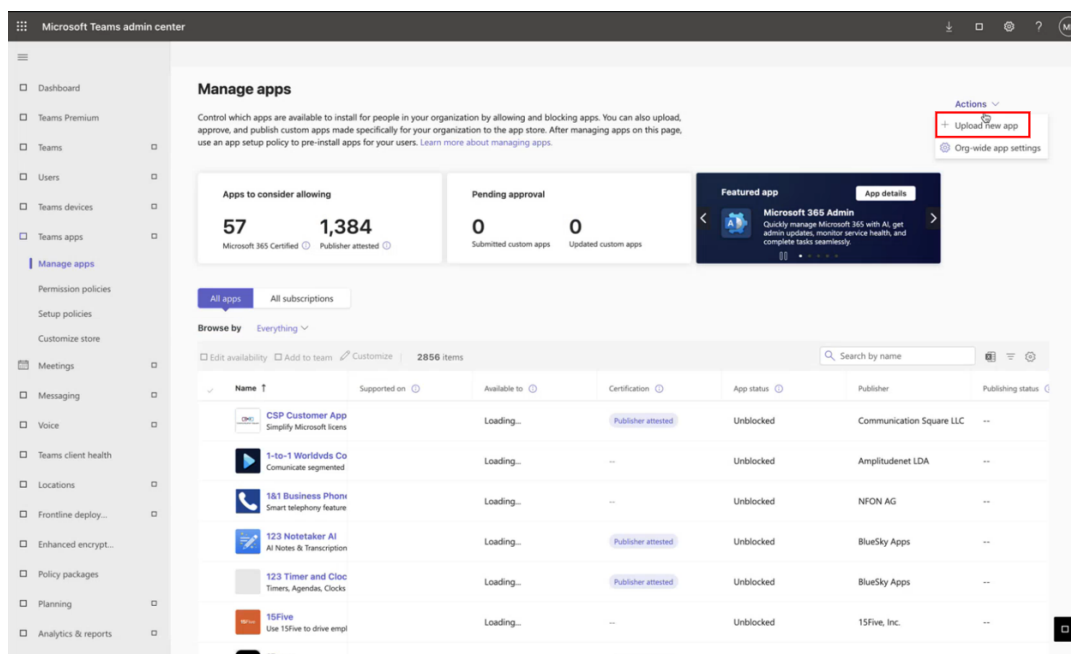
For more information on editing `.env` properties, see [Installing and Configuring the IdentityIQ Service Code](#).

You now have successfully enabled Microsoft Teams notifications. For next step, refer [Installing the IdentityIQ Application in Microsoft Teams](#).

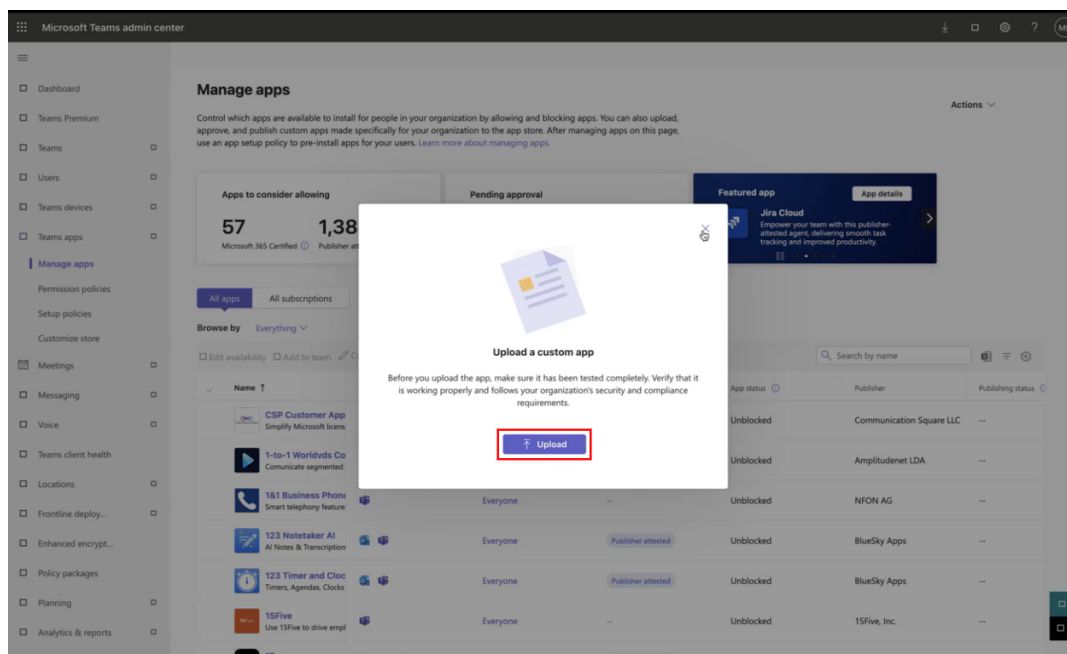
Installing the IdentityIQ Application in Microsoft Teams

Microsoft Teams Administrator must install the IdentityIQ application using the zipped manifest file created in [Creating a Microsoft Teams Manifest](#). The manifest file name will be displayed as `identityiqbot-<version>-manifest.zip`.

1. Launch the [Microsoft Teams admin center](#) in the browser and sign in using the Admin credentials.
2. On the homepage, navigate to **Teams apps > Manage apps**.
3. On Manage apps page, perform the following steps:
 - a. Select **Actions > + Upload new app** at the top right.



- b. On the Upload a custom app dialog, select **Upload**.



- c. Upload the zipped manifest file.
- d. Search the name of the uploaded IdentityIQ application. You will see the application in search result.

Note: By default, the application is **Available to Everyone**, and the **App Status** is **Unblocked**. Admin can change these settings as necessary.

Note: Once the application is published on the Microsoft Teams admin center, users may have to wait for a considerable amount of time before the application is visible on desktop, browser, or mobile devices.

Once the IdentityIQ application is available, individual users must add the application into their Microsoft Teams client. When the application is installed, it will send a notifications command to IdentityIQ. This establishes the connection between the user's Microsoft Teams client and the IdentityIQ server.

Note: If running the initial notifications command does not return the following message *"IdentityIQ notifications have been enabled"*, then wait a few minutes and try running the *notifications* command again manually. You should see expected response once Microsoft Teams is successfully communicating with IdentityIQ.

If, at any point after the application has been deployed in Microsoft Teams, a user stops receiving expected notifications from IdentityIQ, run the *notifications* command from the Microsoft Teams console. If the response reads as *"IdentityIQ notifications have been enabled"*, this confirms that the connection has been successfully restored.

Upgrading IdentityIQ Microsoft Teams

Note: Perform these steps only if you had installed IdentityIQ application in Microsoft Teams during IdentityIQ 8.3 or 8.4 and want to upgrade it to use its latest features.

Latest IdentityIQ application in Microsoft Teams enables the users to receive notifications for Access Request Approval work item and allows them to approve the requests on the application itself.

Use the following information to upgrade IdentityIQ application in Microsoft Team to the latest version. After your upgrade is complete refer [Using IdentityIQ Microsoft Teams](#) for information on the latest features and functionality.

The upgrade process contains the following steps:

1. Uninstall the existing IdentityIQ application in Microsoft Teams.
2. Create a new connector group, if needed. Refer to [Creating a New Connector Group in Azure](#) for detailed instructions.
3. Create an SSO Entra Application Proxy in Azure. Refer to [Creating an SSO Entra Application Proxy in Azure](#) for detailed instructions.
4. Create an Azure Active Directory application in IdentityIQ and configure it with details from various applications. Refer to [Creating an Azure Active Directory Application in IdentityIQ](#) for detailed instructions.
5. Create and configure SSO Provider Application. Refer to [Configuring Single Sign-On to IdentityIQ from Microsoft Teams](#) for detailed instructions.

Note: Creating and configuring of SSO Provider application was optional for previous versions of the IdentityIQ application. Ignore this step, if you had created and configured SSO provide application earlier.

6. Create a Chat Application Proxy in Azure. Refer to [Creating a Chat Application Proxy for IdentityIQ in Azure](#) for detailed instructions.
7. Update Azure Bot by changing the **Messaging Endpoint** and some additional settings in **Add OAuth Connection Settings**. Refer to **Step 6** in [Creating an Azure Bot for IdentityIQ's Microsoft Teams](#) for detailed instructions.
8. Install and configure the IdentityIQ service code. Refer to [Installing and Configuring the IdentityIQ Service Code](#) for detailed instructions.

Note: Reuse the cert directory and BOT_HOME environment that were created during previous configuration.

9. Create a Microsoft Team manifest. Refer to [Creating a Microsoft Teams Manifest](#) for detailed instructions.
10. Enable Microsoft Teams notifications. Refer to [Enabling Microsoft Teams Notifications in IdentityIQ](#) for detailed instructions.
11. Install the latest IdentityIQ application in Microsoft Teams client. Refer to [Installing the IdentityIQ Application in Microsoft Teams](#) for detailed instructions.

Using IdentityIQ Microsoft Teams

Ensure IdentityIQ application is added in Microsoft Teams from the approved apps list.

IdentityIQ application in Microsoft Teams enables the users to receive notifications of all the types available in IdentityIQ and when configured by the users during [Enabling Microsoft Teams Notifications in IdentityIQ](#).

Microsoft Teams users receive notifications from IdentityIQ directly within the Chat tab of their IdentityIQ application. The notifications for Access Request Approval work items allows the users to Approve, Deny, Forward, or Assign/Re-assign the requests on the application itself.

The notification may include links to IdentityIQ, that takes users to specific page for an action within the IdentityIQ. With the Single sign-on (SSO) configured between IdentityIQ and Microsoft Teams, users are taken directly to the relevant page in IdentityIQ without needing to log in again.

This integration is also accessible from Microsoft Teams on mobile phones, tablets and from web browsers. For details on supported operating system (OS) versions and browsers, refer to the SailPoint Installation Guide.

Note: If you're using Firefox or Safari, ensure that pop-ups are allowed and third-party cookies are enabled for smooth access to the IdentityIQ application in Microsoft Teams.

Microsoft Teams users can use these commands:

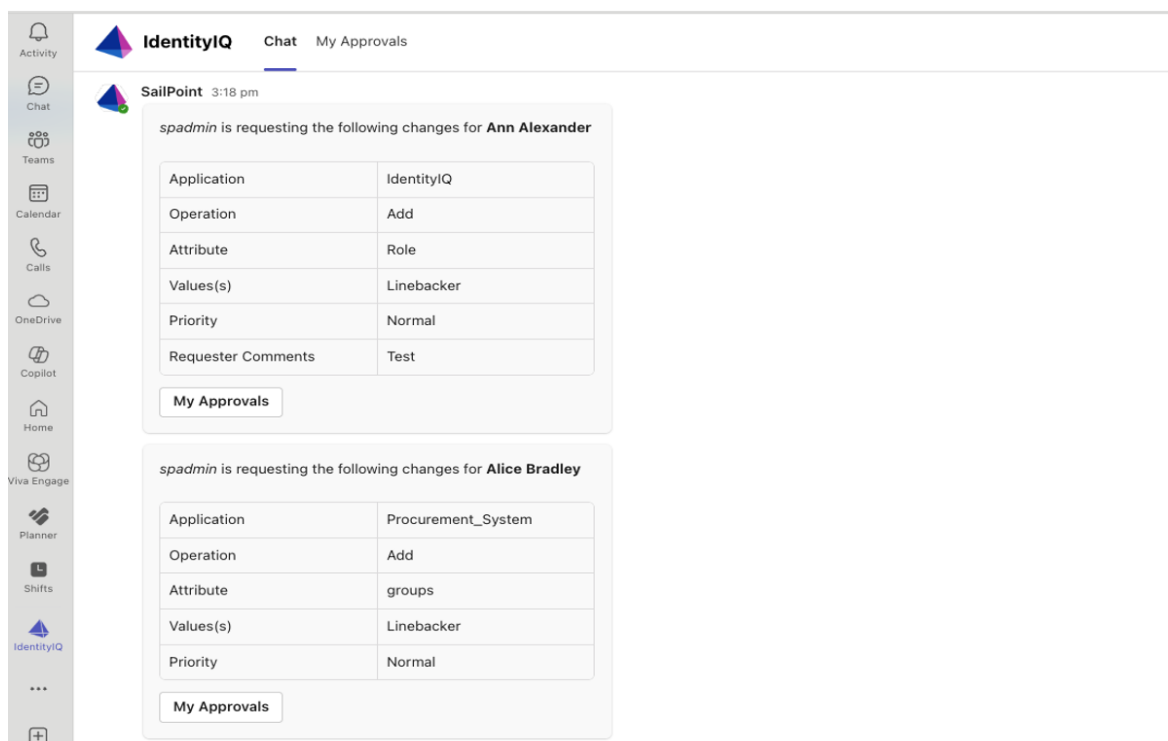
- *hello* – returns a response with the version.
- *help* – displays the bot help message.
- *identityiq* – verifies the connection between IdentityIQ and Microsoft Teams.
- *notifications* – makes the connection between the user's Microsoft Teams environment and their installation of IdentityIQ.

The IdentityIQ application in Microsoft Teams displays two tabs named **Chat** and **My Approvals**.

Note: The name of the My Approvals tab can be customized. You'll see the name that was defined during [Installing and Configuring the IdentityIQ Service Code](#).

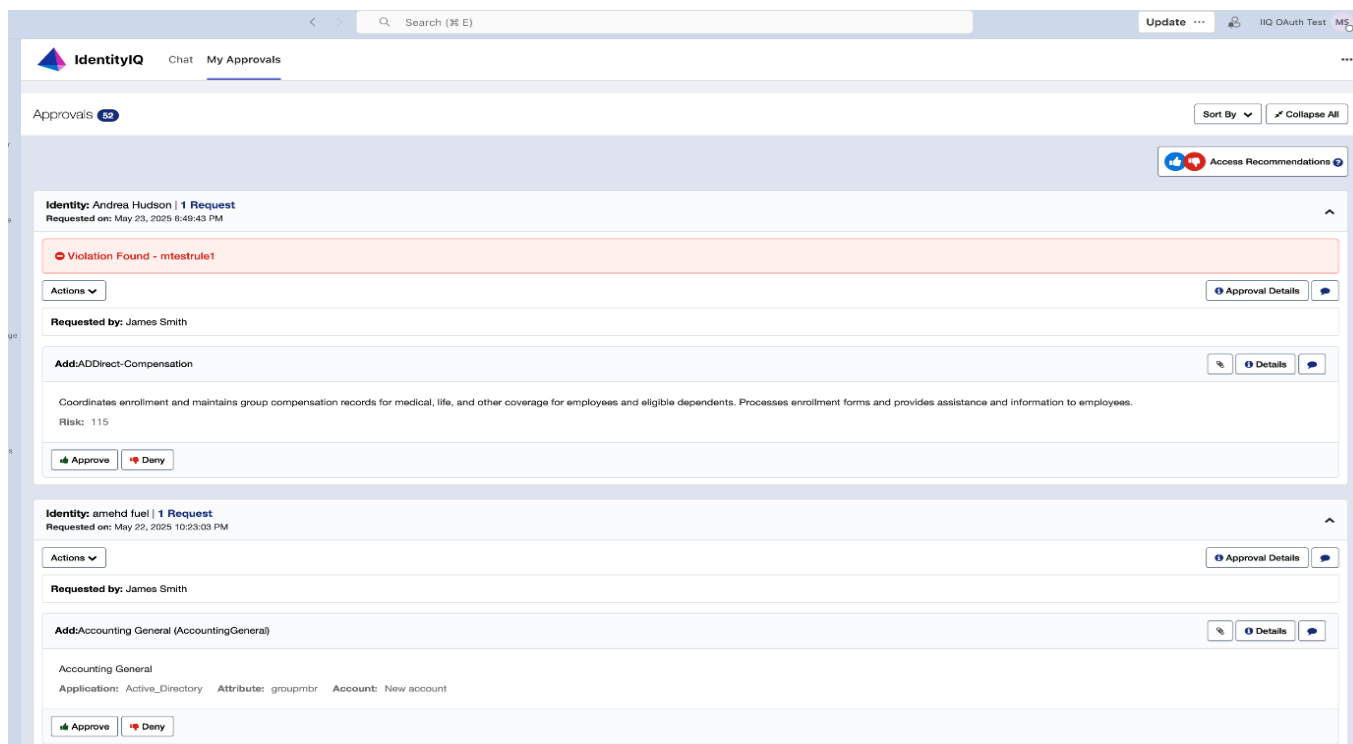
Chat Tab

Notifications will be displayed on the chat tab. Access Request Approval adaptive cards will provide a summary of the request along with a **My Approvals** button for direct access to the respective work item. However, the template used for Email Notifications will not contain the **My Approvals** button as this is only applicable for Adaptive Card Templates.



My Approvals Tab

The *My Approvals* tab displays all the Access Request Approval work items pending for the approver to take action.



Use the IdentityIQ application in Microsoft Teams to make decisions on Access Request Approval work items that are assigned to you. If you are a member of any workgroups, the listings include approvals for those workgroups.

My Approvals tab allows the user to view and manage the Access Request Approval work items. Approval work items sent on Microsoft Teams include the following type of access requests:

- Role Requests
- Entitlement Requests

Approval items are shown in an expanded view by default, showing full details for all items in the request. Select **Collapse All** to switch to a more compact display to view only the high-level work item details.

Select **Expand All** to expand the listing to the detailed view.

Users can perform **Sort** functionality to view the Access Request Approval work items by Newest, Oldest, or Priority.

If you have purchased the SailPoint's AI-Driven Identity Security license and enabled Access Recommendations, a button will be displayed at the top extreme right. See the *SailPoint AI-Driven Identity Security guide* for more information.

Use the **Clip** icon to see the attachment in the Access Request Approval.

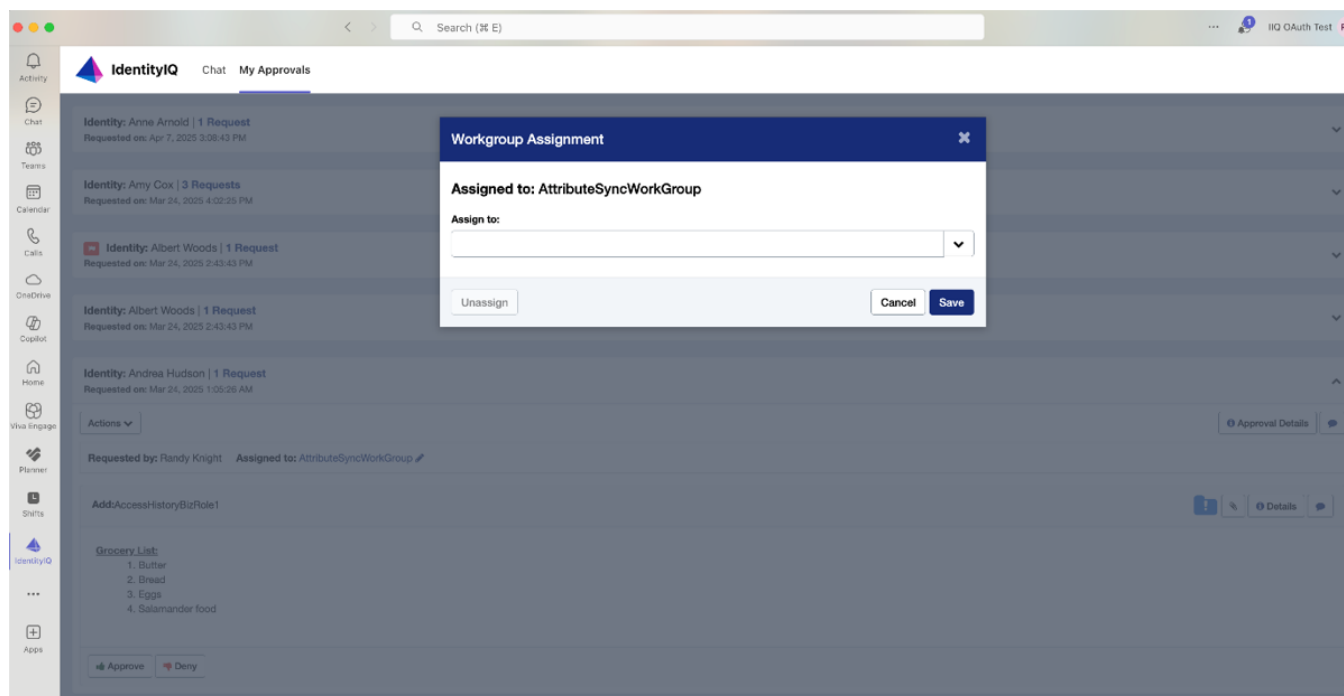
Users can see the details of a request by selecting the **Info** icon. They can also change the start and end date for an Access Request Approval approval request by using the **Calendar** icon.

Approval Tasks on Microsoft Teams

IdentityIQ integration with Microsoft Teams allows users to perform the following tasks directly from within Microsoft Teams either at work items or line items level:

- [Complete an Approval](#)
- [Forward an Approval](#)
- [View Details](#)
- [Viewing Attachments when Approving Requests](#)
- [View and Post Comments](#)

Additionally, members of a workgroup can directly assign or unassign the Access Request Approval work items owned by the group from within Microsoft Teams. To learn more about Workgroups, refer to *SailPoint Roles, Groups, and Populations Guide*.



Auditing Microsoft Teams Notifications

IdentityIQ records the following events and actions performed within the IdentityIQ application in Microsoft Teams in its audit logs:

- Chat Notification Success
- Chat Notification Failure
- Line Item Approval
- Line Item Rejection

- Work Item Assignment
- Work Item Assignment Removal
- Work Item Forwarded

Decisions made on Access Request Approval work items through Microsoft Teams will be recorded in the **Audit Event Details** file. For these work items, a new key:value pair has been added in the logs - Key: Integration and the associated value: Microsoft Teams. This will make it easier for the auditors to identify that the decision taken for the respective work item was made via Microsoft Teams.

Audit Event Details ✕

3/10/25, 2:33 PM

Action Approve Line Item

Source James Smith

Client Host 0:0:0:0:0:0:1

Server Host abolleddu-9H60R4M

Target AlertOwner1

Application IIQ

Account Name

Attribute Name assignedRoles

Attribute Value TestAssignRole2

Attributes	currentStep	Approval
	integration	Microsoft Teams
	interface	LCM
	operation	Add
	requester	spadmin

Close

For more information about auditing in IdentityIQ, see the *SailPoint IdentityIQ System Configuration Guide*.